

FUSION Boot Camp 2026 — Explanatory Notebook Series

29th International Conference on Information Fusion

Trondheim, Norway · June 22–26, 2026

Coordinator: Dale Blair · Instructors: Blasch, Blair, Govaers, Meyer, Bar-Shalom, Varshney

This notebook series documents and demonstrates in code the developments described in FUSION Boot Camp 2026.pdf. The PDF itself contains two complete slide decks — the introduction by Erik Blasch and the Estimation lecture by W. Dale Blair — together with the agenda of the seven Boot Camp lectures.

Boot Camp agenda (22 June 2026)

# Topic	Instructor
1 Introduction to Information Fusion	Erik Blasch
2 Estimation in Information Fusion	Dale Blair
3 Data Association	Florian Meyer
4 Distributed Target Tracking	Felix Govaers
5 Fusion and Association with Attributes/Classification	Yaakov Bar-Shalom
6 Distributed Inference and Information Fusion	Pramod Varshney
7 High-level Fusion	Erik Blasch

Series structure

Notebook	Content	Based on
00_overview.ipynb	Overview, agenda, index	—
01_introduction_to_information_fusion.ipynb	Core concepts, history, JDL/DFIG model (levels 0–5), the Fusion equation, pros/cons, inference hierarchy	Blasch lecture
02_parameter_estimation.ipynb	ML, MAP, MMSE, LSE, WLSE, Cramér–Rao bound, bias estimation	Blair lecture (part 1)
03_state_estimation_kalman_filter.ipynb	Kalman filter derivation, predictor–corrector, process noise DWNA/CWNA, NCV model	Blair lecture (part 2)
04_target_tracking_maneuvering.ipynb	NCV vs NCA, process-noise tuning, IMM, EKF, multiple sensors	Blair lecture (part 3)

Notebook	Content	Based on
05_data_association.ipynb	Gating, GNN, PDA/JPDA, MHT	Meyer lecture*
06_distributed_target_tracking.ipynb	Information filter, track-to-track, Covariance Intersection	Govaers lecture*
07_attributes_classification.ipynb	Sequential Bayesian classification, confusion matrix, Dempster-Shafer	Bar-Shalom lecture*
08_distributed_inference.ipynb	ROC, k-out-of-N rules, Chair-Varshney	Varshney lecture*
09_high_level_fusion.ipynb	Levels L2-L5, situation/threat assessment, MOP/MOE, hard+soft	Blasch lecture (7)*

Note (*): Lectures 3–7 appear in the agenda but their slides are **not** included in the PDF. Notebooks 05–09 are **companion notebooks** based on the canonical literature of the field (Bar-Shalom, Blackman & Popoli, Varshney, Blasch) and on the context Blasch provides in the introduction — not on original slides. Notebooks 01–04 are based directly on the two decks in the PDF.

Requirements

```
pip install numpy matplotlib
```

Each notebook is self-contained and runs top-to-bottom (Run All).

```
# Environment check
import sys
import numpy as np
import matplotlib
print('Python', sys.version.split()[0])
print('numpy', np.__version__)
print('matplotlib', matplotlib.__version__)
```

```
Python 3.11.15
numpy 2.4.6
matplotlib 3.11.0
```

01 · Introduction to Information Fusion¶

Based on: Introduction to Data Fusion — Erik Blasch, FUSION Boot Camp 2026.

Lecture take-aways¶

1. **What is Data Fusion?** — Fusion reduces uncertainty; the quantification of uncertainty is the covariance P .
2. **Methods:** data registration = correlation; data fusion = data association.
3. **What fusion can and cannot do:** it extends space/time coverage and reduces data overload; it is not a 'magic number', does not fix bad data, and does not replace human reasoning.
4. **Architectures:** the JDL/DFIG model, levels 0–5 (taxonomy); multidomain sensing (space, air, cyber) combining hard (images) and soft (text) sources.

A. A brief history of the fusion idea¶

Blasch places information fusion in a long lineage of 'integration of the senses' and inference:

Era	Contribution
Greeks	Plato — propositions/beliefs/evidence; Aristotle — the soul integrates the senses
Materialism	Descartes — body and mind; causal interaction (matter and motion)
1500–1700	Kepler, Galileo, Newton — motion and dynamic interaction
1700	Bayes — beliefs (probability)
1800	Experimentalists — response to a stimulus
1900	Galton, Pearson — correlation (association); William James — integration of senses (FUSE)
1960	Kalman Filter
1990	First set of books on Sensor & Data Fusion

Fusion as a science: three gateways¶

- **Ontology** — what 'the world' is and its constructs (a declaration of what exists; e.g. sensing).
- **Epistemology** — the rules of the game; how we know something (the fusion rules).
- **Taxonomy** — classification/semantics of the information.

B. Types of inference in high-level fusion¶

	Inductive	Deductive	Abductive
Action	from observation to general understanding	derive a consequence from a law	infer the most plausible explanation of an observation
Direction	specific to general	general to specific	incorporate relations

	Inductive	Deductive	Abductive
Methods	Probability, Bayesian, entropy	If-then, hypothesis tests	Bayes Nets

(E. Blasch, P. Valin, E. Bossé, "Measures of Effectiveness for High-Level Fusion," Fusion 2010.)

C. The Key Fusion Equation¶

The central claim: **fusion reduces uncertainty**. When two independent measurements of the same quantity are fused, with variances

$$P_1, P_2$$

(or

$$\sigma_1^2, \sigma_2^2$$

), the fused estimate has a variance **smaller than either**:

$$\frac{1}{P_F} = \frac{1}{P_1} + \frac{1}{P_2} \Rightarrow P_F = \frac{P_1 P_2}{P_1 + P_2}$$

(adapted from Maybeck, 1978). This is exactly the fusion of two Gaussians — equivalent to a Kalman measurement update.

Slide example: with

$$\sigma_1^2 = 1$$

and

$$\sigma_2^2 = 4$$

we get

$$P_F = \frac{1 \cdot 4}{1 + 4} = 0.8$$

, i.e.

$$\sigma_F = \sqrt{0.8} \approx 0.90$$

— smaller than

$$\sigma_1 = 1$$

.

```
import numpy as np
import matplotlib.pyplot as plt

def fuse(x1, P1, x2, P2):
    """Fuse two independent Gaussian estimates (scalar or matrix).
    Returns the fused estimate and the fused covariance."""
    P1, P2 = np.asarray(P1, float), np.asarray(P2, float)
    if P1.ndim == 0: # scalar
        PF = (P1 * P2) / (P1 + P2)
        xF = PF * (x1 / P1 + x2 / P2)
        return xF, PF
    PF = np.linalg.inv(np.linalg.inv(P1) + np.linalg.inv(P2)) # information-form fusion
    xF = PF @ (np.linalg.inv(P1) @ x1 + np.linalg.inv(P2) @ x2)
    return xF, PF
```

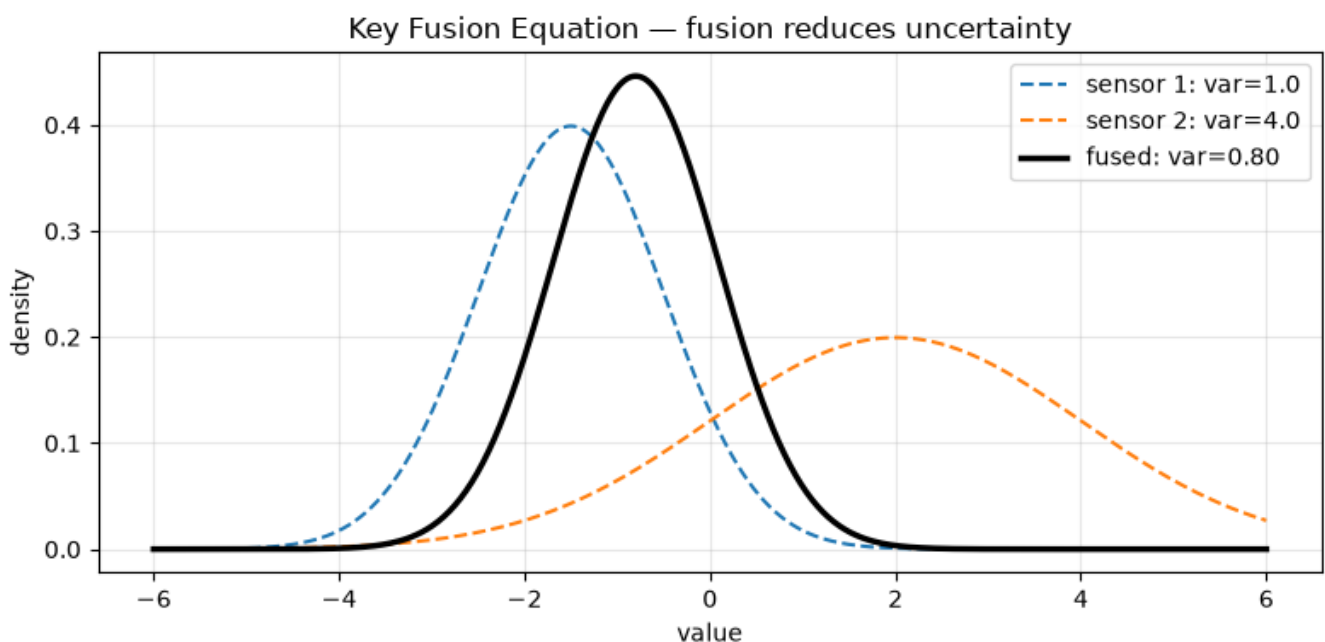
Slide example

```
xF, PF = fuse(0.0, 1.0, 0.0, 4.0)
print(f'P_F = {PF:.3f}, sigma_F = {np.sqrt(PF):.3f} (smaller than sigma_1=1 and sigma_2=2)')
```

P_F = 0.800, sigma_F = 0.894 (smaller than sigma_1=1 and sigma_2=2)

```
# Visualization: two Gaussians and the fused Gaussian
from math import sqrt, pi
x = np.linspace(-6, 6, 400)
def gauss(x, mu, var):
    return np.exp(-(x-mu)**2/(2*var))/sqrt(2*pi*var)

x1, P1, x2, P2 = -1.5, 1.0, 2.0, 4.0
xf, Pf = fuse(x1, P1, x2, P2)
plt.figure(figsize=(8,4))
plt.plot(x, gauss(x, x1, P1), '--', label=f'sensor 1: var={P1}')
plt.plot(x, gauss(x, x2, P2), '--', label=f'sensor 2: var={P2}')
plt.plot(x, gauss(x, xf, Pf), 'k', lw=2.5, label=f'fused: var={Pf:.2f}')
plt.title('Key Fusion Equation — fusion reduces uncertainty')
plt.xlabel('value'); plt.ylabel('density'); plt.legend(); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
print('Note: the fused density is taller and narrower -> smaller variance.')
```



Note: the fused density is taller and narrower -> smaller variance.

Covariance ellipse reduction — the 2-D case [1](#)

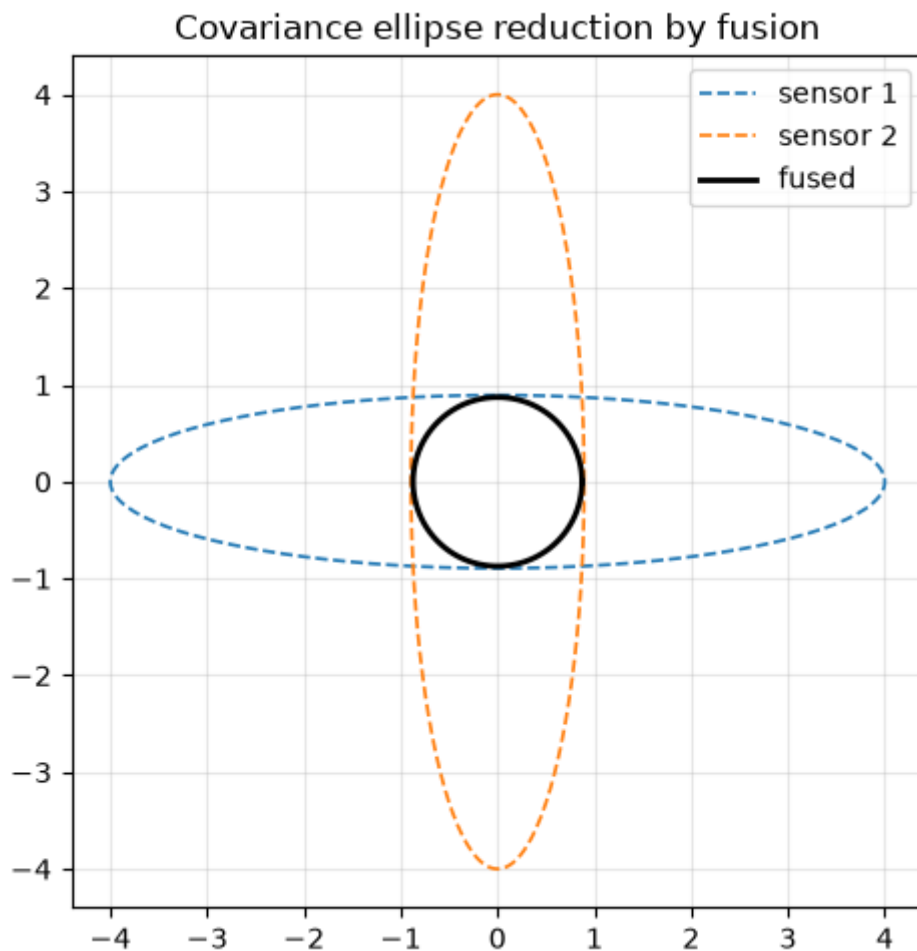
When two sensors measure position in the plane with different error ellipses (e.g. a radar accurate in range but not in angle, and an optical sensor the other way around), fusion shrinks the ellipse area — the very principle behind triangulation.

```
def cov_ellipse(P, mean=(0,0), n_std=2.0, **kw):
    vals, vecs = np.linalg.eigh(P)
    t = np.linspace(0, 2*np.pi, 200)
    circle = np.array([np.cos(t), np.sin(t)])
    ell = vecs @ np.diag(n_std*np.sqrt(vals)) @ circle
```

```
return ell[0]+mean[0], ell[1]+mean[1]
```

```
P1 = np.array([[4.0, 0.0],[0.0, 0.2]]) # accurate along y, coarse along x
P2 = np.array([[0.2, 0.0],[0.0, 4.0]]) # the other way
xF, PF = fuse(np.zeros(2), P1, np.zeros(2), P2)

plt.figure(figsize=(5,5))
for P, lab, st in [(P1,'sensor 1','C0--'),(P2,'sensor 2','C1--'),(PF,'fused','k-')]:
    ex, ey = cov_ellipse(P)
    plt.plot(ex, ey, st, lw=2 if lab=='fused' else 1.2, label=lab)
plt.gca().set_aspect('equal'); plt.legend(); plt.grid(alpha=.3)
plt.title('Covariance ellipse reduction by fusion'); plt.tight_layout(); plt.show()
print('det(P1)=%.3f det(P2)=%.3f det(P_F)=%.3f (fused ellipse area is much smaller)'
      % (np.linalg.det(P1), np.linalg.det(P2), np.linalg.det(PF)))
```



det(P1)=0.800 det(P2)=0.800 det(P_F)=0.036 (fused ellipse area is much smaller)

D. The JDL/DFIG model — fusion levels 0-5 [1](#)

The standard taxonomy for describing fusion functions (originally JDL, extended to DFIG in 2006):

Level Name	Description
L0 Sub-object / Signal	registration and signal processing, feature extraction
L1 Object assessment	

Level Name	Description
	detection, identification (classification) and tracking of objects
L2	Situation assessment
L3	Threat / Impact assessment
L4	Process refinement
L5	User refinement
	estimating relations among entities; the situational picture
	estimating threat, intent, and future consequences
	sensor management and control
	the human in the loop — display, interaction, decision

("DFIG (Data Fusion Information Group), Situation Assessment", J. of Adv. in Info. Fusion, Dec. 2006.)

What we can do today (Can Do): image registration (L0), detection/identification and tracking of non-maneuvering targets in sparse environments (L1), limited automated reasoning at L2/L3, optimization for commensurate sensors at L4, and providing estimates to the user at L5.

What is still hard (Can NOT Do): accurate tracking of maneuvering targets in dense environments, approaching human reasoning ability, accurately modeling intent, and optimizing distributed non-commensurate sensors.

```
# Inference hierarchy – from signal level to knowledge level
levels = [
    ('L0 signal', 'Signal Processing', 'detection, features'),
    ('L1 object', 'Estimation', 'Kalman/Bayesian, MAP'),
    ('L2 situation', 'Decision-level', 'Bayesian nets, fuzzy, NN'),
    ('L3 threat', 'Knowledge-based', 'expert systems, CBR'),
    ('L4/L5 mgmt', 'Control / User', 'sensor mgmt, human-in-loop'),
]
print(f"{'Level':<14}{'Techniques':<22}{'Examples'}")
print('-'*60)
for lv, tech, ex in levels:
    print(f"{'lv':<14}{'tech':<22}{'ex'}")
```

Level	Techniques	Examples
L0 signal	Signal Processing	detection, features
L1 object	Estimation	Kalman/Bayesian, MAP
L2 situation	Decision-level	Bayesian nets, fuzzy, NN
L3 threat	Knowledge-based	expert systems, CBR
L4/L5 mgmt	Control / User	sensor mgmt, human-in-loop

E. Advantages and disadvantages¶

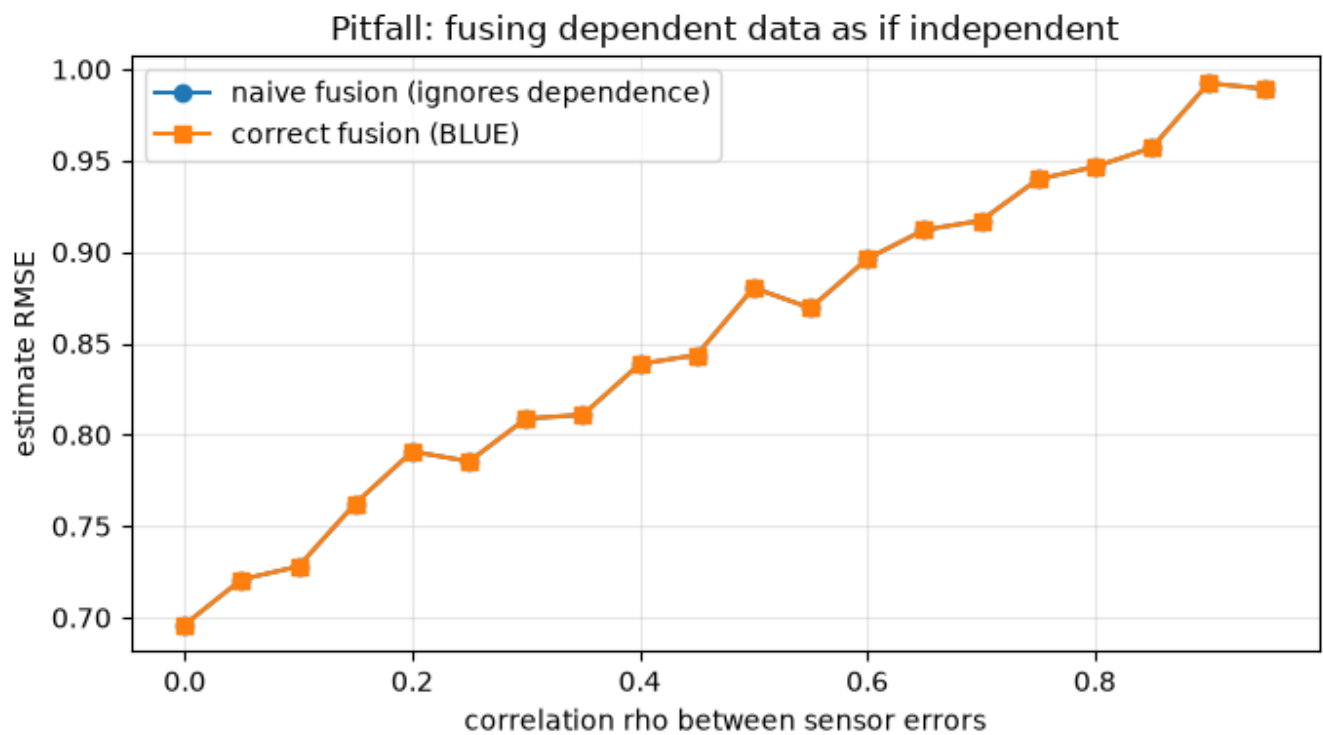
Advantages (Waltz & Llinas, 1990): robust performance (redundancy), extended spatial and temporal coverage, increased confidence and reduced ambiguity, improved detection and resolution, increased dimensionality of the measurement space (resistance to jamming). In short: extends spectral/spatial/temporal coverage and reduces uncertainty and ambiguity; reduces signal-to-noise ratio.

Disadvantages / pitfalls: fusing dependent data (double counting), fusing inconsistent multimodal data, fusing bad data with good, a meaningless fused estimate, over-reliance on 'the number', and not understanding the algorithm's assumptions (most algorithms assume independence).

Data fusion "secrets" (D. Hall): there is no substitute for a good sensor; downstream processing cannot absolve bad upstream processing; the fused answer may be worse than the best sensor; there are no magic algorithms; there will never be enough training data.

```
# Quantitative illustration of a pitfall: fusing two *dependent* measurements as if
independent
# Assume correlation rho between the two sensor errors. The naive formula ignores it.
rng = np.random.default_rng(0)
rho_values = np.linspace(0, 0.95, 20)
true_x = 0.0
s1 = s2 = 1.0 # standard deviations
naive_rmse, correct_rmse = [], []
for rho in rho_values:
    cov = np.array([[s1**2, rho*s1*s2],[rho*s1*s2, s2**2]])
    L = np.linalg.cholesky(cov)
    errs_naive, errs_corr = [], []
    for _ in range(4000):
        e = L @ rng.standard_normal(2)
        z1, z2 = true_x + e
        # naive: ignores correlation -> simple average (equal-variance fusion)
        x_naive = 0.5*z1 + 0.5*z2
        # correct: BLUE with the full covariance matrix
        ones = np.ones(2)
        w = np.linalg.solve(cov, ones); w /= ones @ w
        x_corr = w @ np.array([z1, z2])
        errs_naive.append((x_naive-true_x)**2)
        errs_corr.append((x_corr-true_x)**2)
    naive_rmse.append(np.sqrt(np.mean(errs_naive)))
    correct_rmse.append(np.sqrt(np.mean(errs_corr)))

plt.figure(figsize=(7,4))
plt.plot(rho_values, naive_rmse, 'o-', label='naive fusion (ignores dependence)')
plt.plot(rho_values, correct_rmse, 's-', label='correct fusion (BLUE)')
plt.xlabel('correlation rho between sensor errors'); plt.ylabel('estimate RMSE')
plt.title('Pitfall: fusing dependent data as if independent')
plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```

Summary¹

- **Fusion = uncertainty reduction.** The fusion equation ($1/P_F = 1/P_1 + 1/P_2$) is the core — equivalent to a Kalman measurement update.
- **The JDL/DFIG model** provides a taxonomy (L0–L5) from signal to human-in-the-loop.
- Fusion **extends** coverage and **reduces** uncertainty — but it does **not** create something from nothing, and it depends critically on the independence assumption.

The following notebooks dive into the techniques: **estimation** — the core of level L1.

02 · Parameter Estimation

Based on: Estimation in Information Fusion — W. Dale Blair (GTRI), FUSION Boot Camp 2026, part 1.

Lecture outline

☐ Introduction ☒ **Parametric estimation** ☐ Stochastic state estimation ☐ Target tracking ☐ Concluding remarks

The key distinction

- **Parameter** — a **time-invariant** but unknown quantity (e.g. the position of a stationary target, a sensor bias). We estimate it from noisy measurements.
- **State** — a quantity that **evolves in time** (position/velocity/acceleration), usually linked by differential equations → topic of notebook 03.

General measurement model:

$$z_j = h(x) + w_j$$

, and we seek an estimator

$$\hat{x}(Z_k) \approx x$$

from the measurement set

$$Z_k = \{z_j\}_{j=1}^k$$

.

A. Families of estimators

Estimator	Definition	When to use
ML (Maximum Likelihood)	$\hat{x}_{ML} = \arg \max_x p(Z x)$	no prior information on x
MAP (Maximum A Posteriori)	$\hat{x}_{MAP} = \arg \max_x p(x Z) \propto p(Z x) p(x)$	a prior $p(x)$ is available
MMSE (Min. Mean Squared Error)	$\hat{x}_{MMSE} = E[x Z]$	minimize $E\ x - \hat{x}\ ^2$
LSE (Least Squares)	$\hat{x} = \arg \min_x \sum_j (z_j - h_j(x))^2$	no probabilistic model
WLSE (Weighted LSE)	$\hat{x} = \arg \min_x \sum_j (z_j - h_j(x))^2 / \sigma_j^2$	unequal measurement variances

For independent Gaussian noise, **ML** $\equiv$ **WLSE** (weights =

$$1/\sigma_j^2$$

), and when the variances are equal **ML** $\equiv$ **LSE**.

B. LSE for a linear system

When

$$z = Hx + w$$

(a batch of

N

measurements), the least-squares solution is:

$$\hat{x} = (H^T H)^{-1} H^T z, \quad P_{\hat{x}} = \sigma_w^2 (H^T H)^{-1}$$

and for WLSE with a measurement covariance matrix

R

:

$$\hat{x} = (H^T R^{-1} H)^{-1} H^T R^{-1} z, \quad P_{\hat{x}} = (H^T R^{-1} H)^{-1}$$

Slide example: 11 measurements of a stationary target with

$$\sigma_w = 2$$

m. Here

$$h(x) = x$$

, i.e.

$$H = \mathbf{1}$$

, and the solution reduces to the **mean**:

$$\hat{x} = \frac{1}{N} \sum z_j$$

with variance

$$\sigma_w^2 / N$$

.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(1)

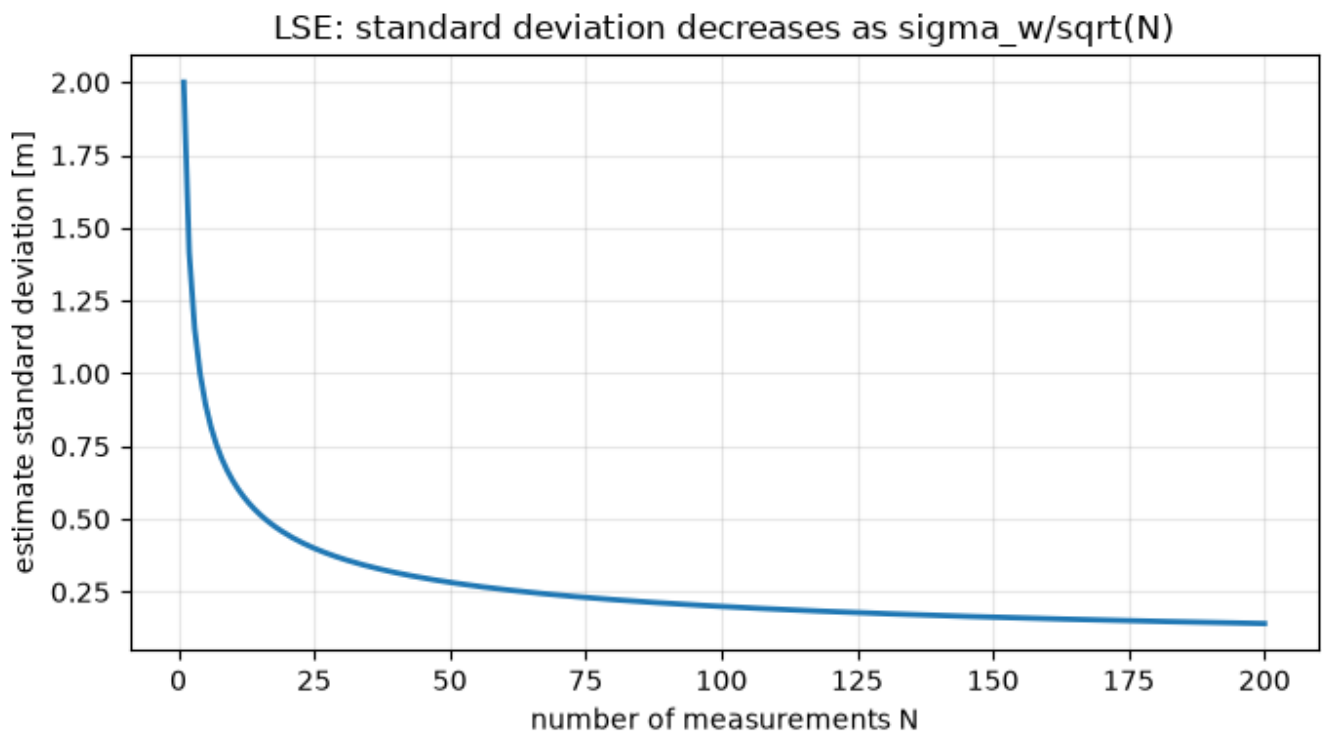
def lse_linear(H, z, R=None):
    """LSE / WLSE for a linear system z = H x + w. R = covariance matrix (optional)."""
    H = np.atleast_2d(H); z = np.asarray(z, float).reshape(-1, 1)
    if R is None:
        G = np.linalg.inv(H.T @ H) @ H.T
        xhat = G @ z
        return xhat.ravel(), np.linalg.inv(H.T @ H) # multiply by sigma^2 for P
    Ri = np.linalg.inv(R)
    P = np.linalg.inv(H.T @ Ri @ H)
    xhat = P @ H.T @ Ri @ z
    return xhat.ravel(), P

# Stationary-target example (11 measurements, sigma_w = 2)
true_x = 100.0; sigma_w = 2.0; N = 11
z = true_x + sigma_w * rng.standard_normal(N)
H = np.ones((N, 1))
```

```
xhat, M = lse_linear(H, z)
P = sigma_w**2 * M
print(f'LSE estimate = {xhat[0]:.3f} m    (truth = {true_x})')
print(f'estimate variance = {P[0,0]:.3f} = sigma_w^2/N = {sigma_w**2/N:.3f}')
print(f'standard deviation = {np.sqrt(P[0,0]):.3f} m')
```

LSE estimate = 100.414 m (truth = 100.0)
 estimate variance = 0.364 = sigma_w^2/N = 0.364
 standard deviation = 0.603 m

```
# Convergence: the standard deviation decreases as 1/sqrt(N)
Ns = np.arange(1, 201)
std_theory = sigma_w / np.sqrt(Ns)
plt.figure(figsize=(7,4))
plt.plot(Ns, std_theory, 'C0', lw=2)
plt.xlabel('number of measurements N'); plt.ylabel('estimate standard deviation [m]')
plt.title('LSE: standard deviation decreases as sigma_w/sqrt(N)'); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
```



C. The Cramér-Rao Lower Bound (CRLB)¶

The CRLB is a lower bound on the variance of **any** unbiased estimator:

$$\text{Var}(\hat{x}) \geq \frac{1}{I(x)}, \quad I(x) = -E \left[\frac{\partial^2 \ln p(Z|x)}{\partial x^2} \right]$$

where

$$I(x)$$

is the **Fisher Information**. An estimator that attains the bound is called efficient. For

N

independent Gaussian measurements of a stationary parameter,

$$I(x) = N/\sigma_w^2$$

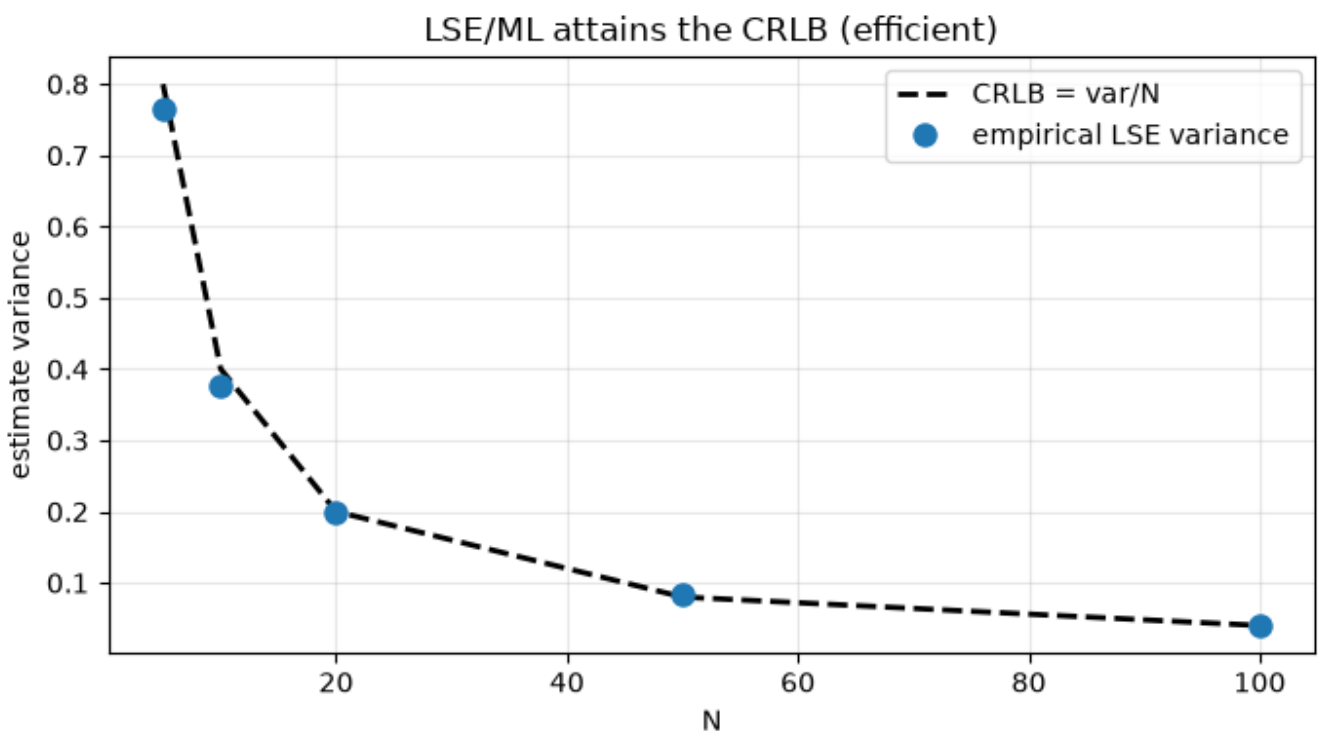
, so

$$\text{Var}(\hat{x}) \geq \sigma_w^2/N$$

— and LSE/ML attains equality (efficient).

```
# Monte-Carlo verification: does LSE reach the CRLB?
sigma_w = 2.0
Ns = [5, 10, 20, 50, 100]
mc = 5000
emp_var, crlb = [], []
for N in Ns:
    ests = []
    for _ in range(mc):
        z = true_x + sigma_w*rng.standard_normal(N)
        ests.append(z.mean())
    emp_var.append(np.var(ests))
    crlb.append(sigma_w**2 / N)

plt.figure(figsize=(7,4))
plt.plot(Ns, crlb, 'k--', lw=2, label='CRLB = var/N')
plt.plot(Ns, emp_var, 'o', ms=8, label='empirical LSE variance')
plt.xlabel('N'); plt.ylabel('estimate variance'); plt.legend(); plt.grid(alpha=.3)
plt.title('LSE/ML attains the CRLB (efficient)'); plt.tight_layout(); plt.show()
```



D. ML estimation — a nonlinear example

Blair demonstrates an ML estimator on a nonlinear model of the form

$$z_j = \cos(2\pi wx) + w_j$$

. Here the likelihood is non-convex and the maximum must be found numerically (grid search / optimization). Below we use a simpler nonlinear model to estimate a **frequency**

f

from a sampled signal:

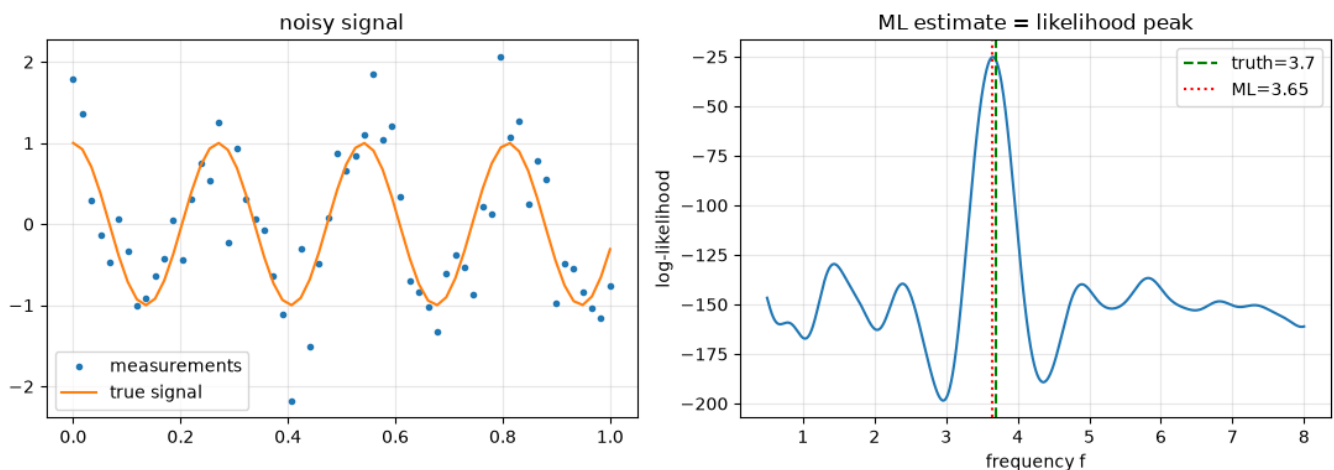
$$z_j = \cos(2\pi f t_j) + w_j$$

.

```
# ML frequency estimation by scanning the log-likelihood
f_true = 3.7; sigma = 0.5
t = np.linspace(0, 1, 60)
z = np.cos(2*np.pi*f_true*t) + sigma*rng.standard_normal(t.size)

f_grid = np.linspace(0.5, 8, 1500)
# Gaussian log-likelihood: -0.5/sigma^2 * sum (z - cos(2 pi f t))^2 (+const)
ll = np.array([-0.5/sigma**2 * np.sum((z - np.cos(2*np.pi*f*t))**2) for f in f_grid])
f_ml = f_grid[np.argmax(ll)]

fig, ax = plt.subplots(1, 2, figsize=(11,4))
ax[0].plot(t, z, '.', label='measurements'); ax[0].plot(t, np.cos(2*np.pi*f_true*t),
label='true signal')
ax[0].set_title('noisy signal'); ax[0].legend(); ax[0].grid(alpha=.3)
ax[1].plot(f_grid, ll); ax[1].axvline(f_true, color='g', ls='--', label=f'truth={f_true}')
ax[1].axvline(f_ml, color='r', ls=':', label=f'ML={f_ml:.2f}')
ax[1].set_xlabel('frequency f'); ax[1].set_ylabel('log-likelihood'); ax[1].set_title('ML
estimate = likelihood peak')
ax[1].legend(); ax[1].grid(alpha=.3); plt.tight_layout(); plt.show()
print(f'ML frequency estimate = {f_ml:.3f} (truth = {f_true})')
```



ML frequency estimate = 3.652 (truth = 3.7)

E. Sensor bias estimation and observability [1](#)

Blair devotes several slides to estimating a sensor **bias**. The key lesson: with a single sensor, position

x

and bias

b

are **not separately observable** — the measurement is always

$$z = x + b + w$$

, and any split between

$$x$$

and

$$b$$

fits the same data. The matrix

$$H^T H$$

becomes singular and "the estimate cannot be computed".

The remedy: adding another information source (a second sensor, or a calibration/reference target whose position is known) makes the problem observable — and this is exactly where **fusion** enters: combining sources resolves unobservability.

```
# Demonstration: single sensor -> H^T H singular (x and b not separately observable)
# Measurement: z = x + b + w. Unknowns: [x, b].
N = 10
H_single = np.hstack([np.ones((N,1)), np.ones((N,1))]) # the two columns are identical!
print('rank(H) (single sensor):', np.linalg.matrix_rank(H_single), 'of 2 -> singular, cannot
separate x and b')

# Remedy: a calibration target at known position x0=0 (b alone observable) + an unknown target
# the sensor measures the calibration target and the target: z_cal = 0 + b + w ; z_tgt = x + b
+ w
x_true, b_true, sw = 50.0, 4.0, 1.0
m = 200
z_cal = 0 + b_true + sw*rng.standard_normal(m)
z_tgt = x_true + b_true + sw*rng.standard_normal(m)
# unknowns theta=[x, b]; two equations: z_cal = b ; z_tgt = x + b
Hc = np.array([[0.,1.],[1.,1.]])
zc = np.array([z_cal.mean(), z_tgt.mean()])
theta, _ = lse_linear(Hc, zc, R=np.eye(2)*(sw**2/m))
print(f'with a calibration target: x_hat = {theta[0]:.2f} (truth {x_true}), b_hat =
{theta[1]:.2f} (truth {b_true})')
print('-> an additional information source (fusion) made the problem observable.')
```

rank(H) (single sensor): 1 of 2 -> singular, cannot separate x and b
 with a calibration target: x_hat = 50.00 (truth 50.0), b_hat = 3.90 (truth 4.0)
 -> an additional information source (fusion) made the problem observable.

Summary1

- **ML / MAP / MMSE / LSE / WLSE** — the family of estimators; under Gaussian noise they coincide (ML≡WLSE, and equal-variance ⇒ ≡LSE).
- **Linear LSE:**

$$\hat{x} = (H^T R^{-1} H)^{-1} H^T R^{-1} z$$

with variance

$$(H^T R^{-1} H)^{-1}$$

.

- **The CRLB** sets the floor on variance; LSE/ML is efficient for a stationary target.
- **Observability:** bias and position are inseparable for a single sensor — **fusion** of sources resolves this.

In notebook 03 we move from a **stationary** parameter to a state that **evolves in time** — leading to the Kalman filter.

03 · Stochastic State Estimation and the Kalman Filter

Based on: Estimation in Information Fusion — W. Dale Blair, FUSION Boot Camp 2026, part 2.

In notebook 02 we estimated a **stationary** parameter. Now the state **evolves in time** (position, velocity, acceleration). **The Kalman filter is the 'workhorse' of target tracking** and gives the MMSE estimate when the errors are Gaussian.

A. The discrete-time model

$$x_{k+1} = F_k x_k + G_k v_k, \quad z_k = H_k x_k + w_k$$

- x_k
— the state vector (position, velocity, possibly acceleration).
- $v_k \sim \mathcal{N}(0, Q_k)$
— **process noise** (plant noise).
- $w_k \sim \mathcal{N}(0, R_k)$
— measurement noise.
- F
— the transition matrix (the kinematic motion constraint);
- H
— relates measurements to the state (nonlinear forms → EKF, notebook 04).

Process noise is the essential difference between the Kalman filter (stochastic state estimation) and LSE (parameter estimation): it captures the uncertainty in the evolution/ modeling of the state from step

k
to
 $k+1$
, zero-mean and white.

B. Deriving the Kalman filter — predictor / corrector

The filter is a **predictor-corrector** algorithm:

Time Update (Predict):

$$\hat{x}_{k|k-1} = F_{k-1} \hat{x}_{k-1|k-1}, \quad P_{k|k-1} = F_{k-1} P_{k-1|k-1} F_{k-1}^T + G_{k-1} Q_{k-1} G_{k-1}^T$$

Measurement Update (Correct):

$$S_k = H_k P_{k|k-1} H_k^T + R_k \quad (\text{innovation covariance})$$

$$K_k = P_{k|k-1} H_k^T S_k^{-1} \quad (\text{Kalman gain})$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k [z_k - H_k \hat{x}_{k|k-1}]$$

$$P_{k|k} = (I - K_k H_k) P_{k|k-1}$$

The gain

balances trust in the model (predict) against trust in the measurement: large

→ small

(trust the model), and vice versa.

Slide warning: the Kalman filter assumes **perfect measurement-to-track association** — exactly where data association enters (Meyer's lecture).

C. The NCV model and process noise (DWNA vs CWNA)¶

Nearly Constant Velocity (NCV) — the most widely used model in tracking. State vector

$$x = [p, \dot{p}]^T$$

:

$$F = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Two common ways to model the process noise (unplanned acceleration with variance

or PSD

):

DWNA — Discrete White Noise Acceleration:

$$Q = \Gamma \Gamma^T \sigma_a^2, \quad \Gamma = \begin{bmatrix} T^2/2 \\ T \end{bmatrix} \Rightarrow Q = \sigma_a^2 \begin{bmatrix} T^4/4 & T^3/2 \\ T^3/2 & T^2 \end{bmatrix}$$

CWNA — Continuous White Noise Acceleration:

$$Q = q \begin{bmatrix} T^3/3 & T^2/2 \\ T^2/2 & T \end{bmatrix}$$

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(7)

def ncv_matrices(T, kind='DWNA', sigma_a=1.0):
    F = np.array([[1, T], [0, 1.0]])
    H = np.array([[1.0, 0.0]])
    if kind == 'DWNA':
        G = np.array([[T**2/2], [T]])
        Q = (G @ G.T) * sigma_a**2
    else: # CWNA
        Q = sigma_a**2 * np.array([[T**3/3, T**2/2], [T**2/2, T]])
    return F, H, Q

class KalmanFilter:
    def __init__(self, F, H, Q, R, x0, P0):
        self.F, self.H, self.Q, self.R = map(np.atleast_2d, (F, H, Q, R))
        self.x = np.asarray(x0, float).reshape(-1, 1)
        self.P = np.atleast_2d(P0).astype(float)
    def predict(self):
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.x.ravel()
    def update(self, z):
        z = np.atleast_2d(np.asarray(z, float).reshape(-1, 1))
        S = self.H @ self.P @ self.H.T + self.R
        K = self.P @ self.H.T @ np.linalg.inv(S)
        y = z - self.H @ self.x
        self.x = self.x + K @ y
        I = np.eye(self.P.shape[0])
        self.P = (I - K @ self.H) @ self.P
        return self.x.ravel(), y.item(), S.item()

print('Kalman filter + NCV models defined.')
```

Kalman filter + NCV models defined.

D. Simulation: NCV tracking of a constant-velocity target¹

A target moves at constant velocity; the sensor measures **only position** with noise

$$\sigma_w$$

. The filter estimates position and **velocity** (which is not measured directly) — the power of state estimation.

```

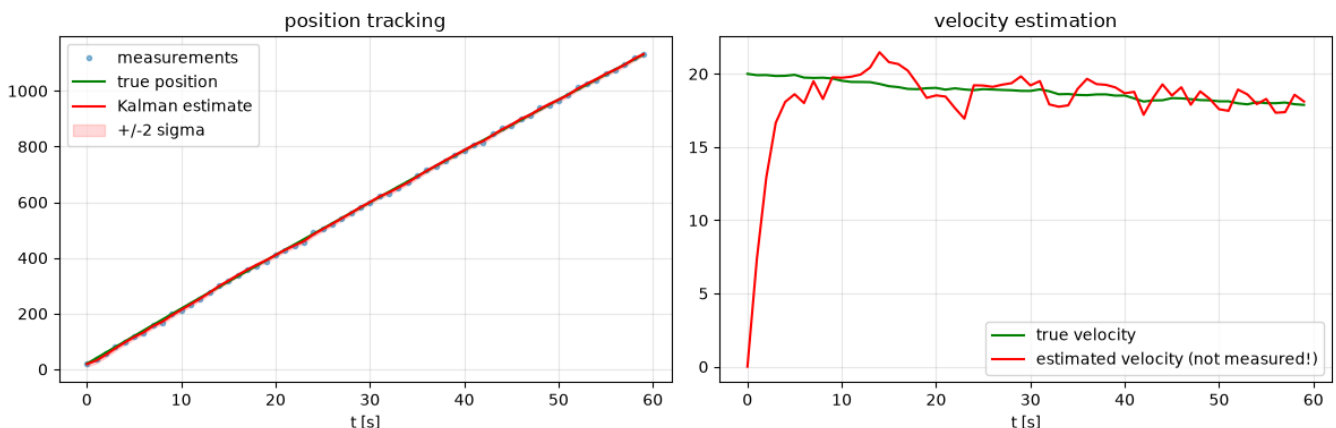
T, steps = 1.0, 60
sigma_w = 5.0          # measurement noise [m]
sigma_a_true = 0.1     # small true maneuver
F, H, _ = ncv_matrices(T)
Qsim = ncv_matrices(T, 'DWNA', sigma_a_true)[2]

# generate the ground-truth trajectory
x = np.array([[0.],[20.]]) # position 0, velocity 20 m/s
truth, meas = [], []
Lq = np.linalg.cholesky(Qsim + 1e-12*np.eye(2))
for _ in range(steps):
    x = F @ x + Lq @ rng.standard_normal((2,1))
    truth.append(x.ravel().copy())
    meas.append((H @ x).item() + sigma_w*rng.standard_normal())
truth = np.array(truth); meas = np.array(meas)

# run the filter
Fk, Hk, Qk = ncv_matrices(T, 'DWNA', sigma_a=1.0)
kf = KalmanFilter(Fk, Hk, Qk, R=sigma_w**2, x0=[meas[0],0], P0=np.diag([sigma_w**2,100]))
est_pos, est_vel, pos_std = [], [], []
for z in meas:
    kf.predict()
    xe,_,_ = kf.update(z)
    est_pos.append(xe[0]); est_vel.append(xe[1]); pos_std.append(np.sqrt(kf.P[0,0]))
est_pos,est_vel,pos_std = map(np.array,(est_pos,est_vel,pos_std))

t = np.arange(steps)*T
fig, ax = plt.subplots(1,2,figsize=(12,4))
ax[0].plot(t, meas,'.',alpha=.5,label='measurements')
ax[0].plot(t, truth[:,0],'g',label='true position')
ax[0].plot(t, est_pos,'r',label='Kalman estimate')
ax[0].fill_between(t, est_pos-2*pos_std, est_pos+2*pos_std, color='r', alpha=.15, label='+/-2 sigma')
ax[0].set_title('position tracking'); ax[0].legend(); ax[0].grid(alpha=.3);
ax[0].set_xlabel('t [s]')
ax[1].plot(t, truth[:,1],'g',label='true velocity')
ax[1].plot(t, est_vel,'r',label='estimated velocity (not measured!)')
ax[1].set_title('velocity estimation'); ax[1].legend(); ax[1].grid(alpha=.3);
ax[1].set_xlabel('t [s]')
plt.tight_layout(); plt.show()
print(f'position RMSE: raw={np.sqrt(np.mean((meas-truth[:,0])**2)):.2f} m, '
      f' filtered={np.sqrt(np.mean((est_pos-truth[:,0])**2)):.2f} m')

```



position RMSE: raw=4.35 m, filtered=3.41 m

E. The role of process noise and gain convergence [1](#)

When

Q

and

R

are constant, the covariance

P

and the gain

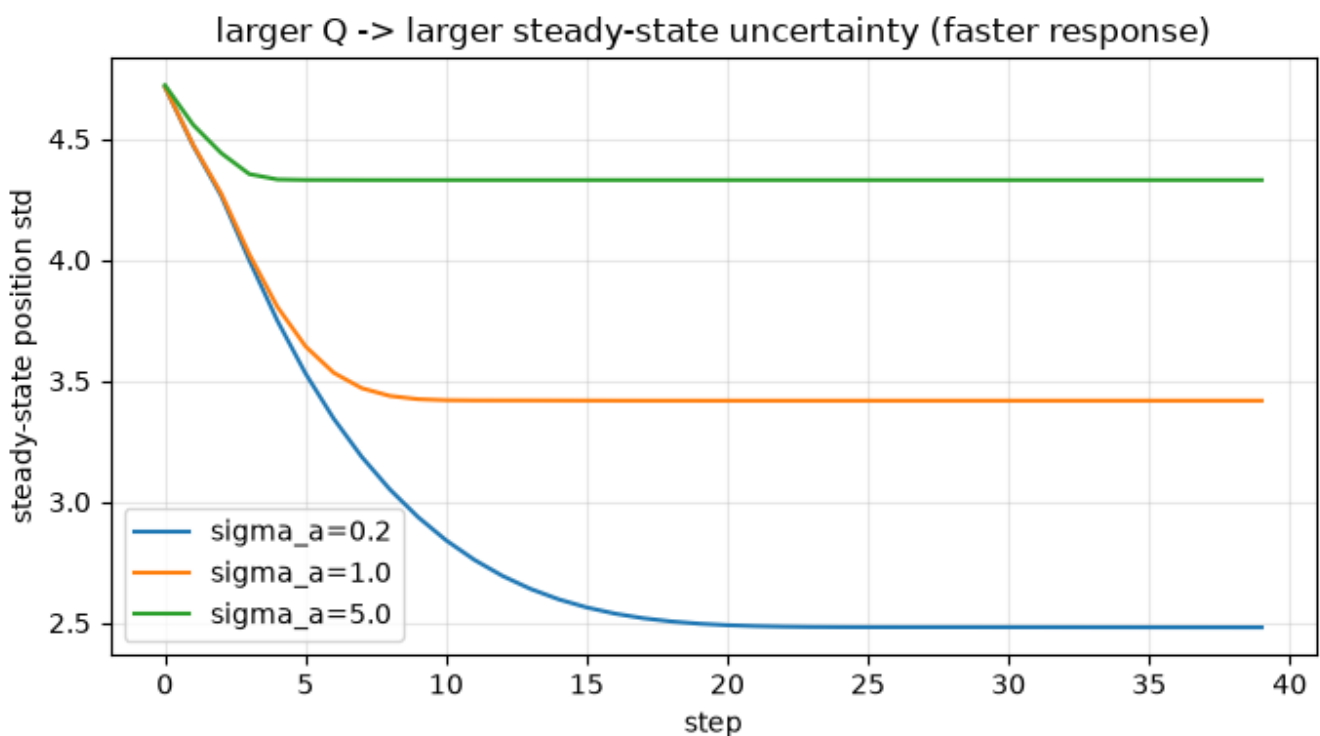
K

converge to a steady state. The smaller

Q

is, the more the filter 'trusts' the model, the smaller the gain, and the smoother but slower-to-maneuver the estimate (notebook 04).

```
# gain / position-std convergence for different sigma_a values
plt.figure(figsize=(7,4))
for sa in [0.2, 1.0, 5.0]:
    Fk,Hk,Qk = ncv_matrices(T,'DWNA',sigma_a=sa)
    kf = KalmanFilter(Fk,Hk,Qk,R=sigma_w**2,x0=[0,0],P0=np.diag([100.,100.]))
    stds=[]
    for _ in range(40):
        kf.predict(); kf.update(0.0); stds.append(np.sqrt(kf.P[0,0]))
    plt.plot(stds, label=f'sigma_a={sa}')
plt.xlabel('step'); plt.ylabel('steady-state position std')
plt.title('larger Q -> larger steady-state uncertainty (faster response)')
plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



F. Consistency check (normalized innovation)

A consistent filter produces innovations that match the covariance

$$S_k$$

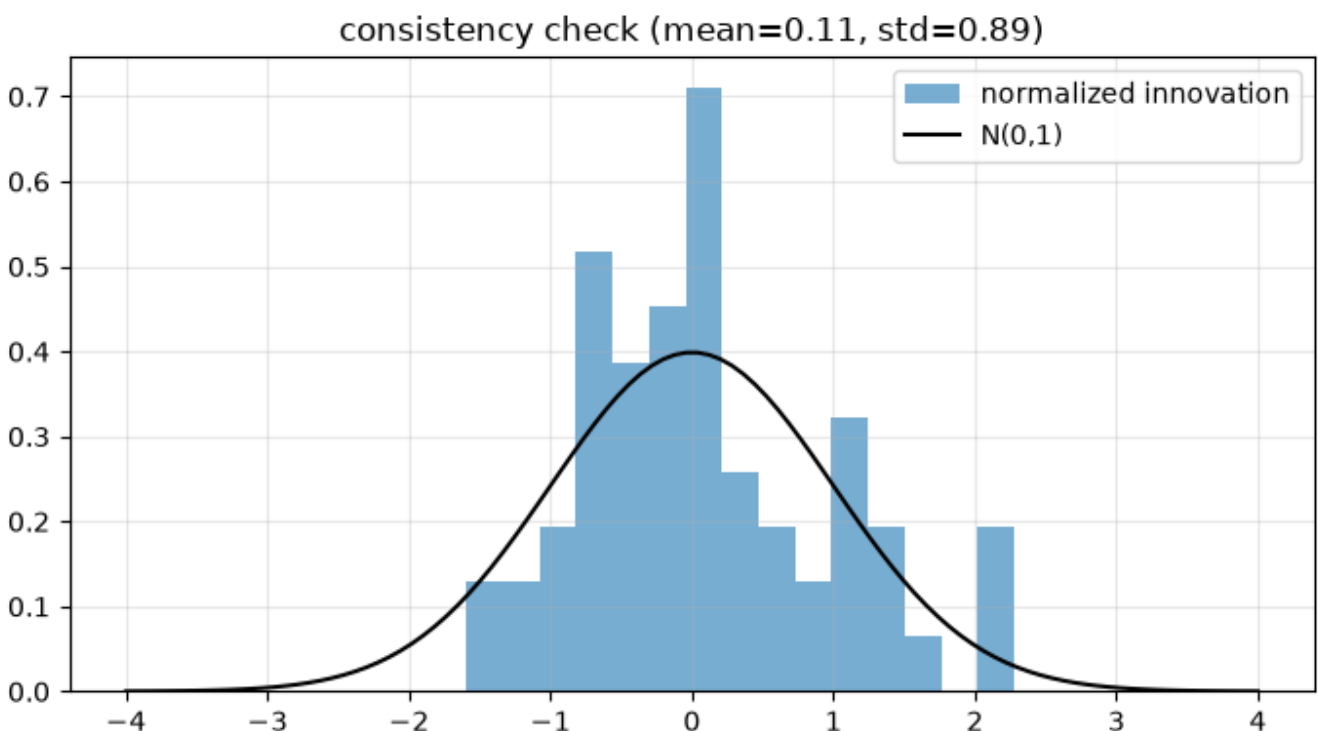
it predicts. A common diagnostic: the normalized innovation

$$y_k/\sqrt{S_k}$$

should be distributed as

$$\mathcal{N}(0,1)$$

```
kf = KalmanFilter(Fk:=ncv_matrices(T,'DWNA',1.0)[0], ncv_matrices(T)[1],
                  ncv_matrices(T,'DWNA',1.0)[2], R=sigma_w**2,
                  x0=[meas[0],0], P0=np.diag([sigma_w**2,100]))
norm_innov=[]
for z in meas:
    kf.predict(); _,y,S = kf.update(z); norm_innov.append(y/np.sqrt(S))
norm_innov=np.array(norm_innov)
plt.figure(figsize=(7,4))
plt.hist(norm_innov, bins=15, density=True, alpha=.6, label='normalized innovation')
xx=np.linspace(-4,4,100); plt.plot(xx, np.exp(-xx**2/2)/np.sqrt(2*np.pi),'k',label='N(0,1)')
plt.title(f'consistency check (mean={norm_innov.mean():.2f}, std={norm_innov.std():.2f})')
plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



Summary

- **The Kalman filter = predictor-corrector**; gives the MMSE under linearity + Gaussianity.
- **Process noise** (

$$Q$$

, in DWNA/CWNA form) distinguishes state estimation from parameter estimation and controls the model↔measurement balance.

- The **NCV** model estimates velocity even when only position is measured, and significantly reduces the position error relative to the raw measurement.
- The filter assumes **perfect association** and a **single** model — two assumptions relaxed in notebook 04 (maneuvers, IMM, nonlinearity).

04 · Tracking Maneuvering Targets: NCV/ NCA, IMM and EKF

Based on: Estimation in Information Fusion — W. Dale Blair, FUSION Boot Camp 2026, part 3.

In notebook 03 we built a Kalman filter with a **single** motion model (NCV). Real targets **maneuver**, raising three issues: model selection (NCV vs NCA), process-noise tuning, and combining several models (IMM). In addition, real measurements (range/bearing) are **nonlinear** → EKF.

A. NCV vs NCA

- **NCV** (Nearly Constant Velocity): state

$$[p, \dot{p}]$$

. Simple; excellent for straight-line motion; **lags in a maneuver.**

- **NCA** (Nearly Constant Acceleration): state

$$[p, \dot{p}, \ddot{p}]$$

. Tracks acceleration; but 'noisier' in quiet motion (estimates acceleration from noise).

NCA matrices (DWNA):

$$F = \begin{bmatrix} 1 & T & T^2/2 \\ 0 & 1 & T \\ 0 & 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}, \quad Q = \sigma_a^2 \Gamma \Gamma^T, \quad \Gamma = \begin{bmatrix} T^2/2 \\ T \\ 1 \end{bmatrix}$$

The dilemma (from the slides): small

$$\sigma_a$$

→ small errors in quiet motion but a large lag in a maneuver; large

$$\sigma_a$$

→ minimizes the maximum error in a maneuver but increases the average error. **There is no single optimal choice** — the motivation for IMM.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(11)

def ncv(T, sa):
    F=np.array([[1,T],[0,1]]); H=np.array([[1,0]])
    G=np.array([[T**2/2],[T]]); Q=(G@G.T)*sa**2
    return F,H,Q
def nca(T, sa):
    F=np.array([[1,T,T**2/2],[0,1,T],[0,0,1]]); H=np.array([[1,0,0]])
    G=np.array([[T**2/2],[T],[1]]); Q=(G@G.T)*sa**2
    return F,H,Q
```



```

class KF:
    def __init__(s,F,H,Q,R,x,P): s.F,s.H,s.Q,s.R=map(np.atleast_2d,(F,H,Q,R));
    s.x=np.array(x,float).reshape(-1,1); s.P=np.array(P,float)
    def predict(s): s.x=s.F@s.x; s.P=s.F@s.P@s.F.T+s.Q
    def update(s,z):
        S=s.H@s.P@s.H.T+s.R; K=s.P@s.H.T@np.linalg.inv(S)
        y=np.atleast_2d(z).reshape(-1,1)-s.H@s.x
        s.x=s.x+K@y; s.P=(np.eye(s.P.shape[0])-K@s.H@s.P
        d=float((y.T@np.linalg.inv(S)@y).item())
        L=float(np.exp(-0.5*d)/np.sqrt(np.linalg.det(2*np.pi*S)))
        return L
print('NCV/NCA + KF defined.')

```

NCV/NCA + KF defined.

```

# ground-truth trajectory with a maneuver: constant velocity -> accel 40-60s -> constant again
T=1.0; N=100; sw=10.0
t=np.arange(N)*T
acc=np.where((t>=40)&(t<60), 8.0, 0.0) # acceleration m/s^2 in the maneuver segment
pos=np.zeros(N); vel=np.zeros(N); vel[0]=30
for k in range(1,N):
    vel[k]=vel[k-1]+acc[k-1]*T
    pos[k]=pos[k-1]+vel[k-1]*T+0.5*acc[k-1]*T**2
meas=pos+sw*rng.standard_normal(N)

def run_single(make, sa, dim):
    F,H,Q=make(T,sa); P0=np.diag([sw**2,100,100][:dim])
    x0=[meas[0]]+[0]*(dim-1)
    kf=KF(F,H,Q,sw**2,x0,P0); est=[]
    for z in meas: kf.predict(); kf.update(z); est.append(kf.x[0,0])
    return np.array(est)

est_ncv_lo=run_single(ncv,0.5,2)
est_ncv_hi=run_single(ncv,8.0,2)
est_nca =run_single(nca,8.0,3)
for name,e in [('NCV sigma_a=0.5',est_ncv_lo),('NCV sigma_a=8',est_ncv_hi),('NCA
sigma_a=8',est_nca)]:
    print(f'{name:16s} RMSE={np.sqrt(np.mean((e-pos)**2)):6.2f} m '
          f'maneuver RMSE={np.sqrt(np.mean((e[40:60]-pos[40:60])**2)):6.2f} m')

```

NCV sigma_a=0.5	RMSE= 55.84 m	maneuver RMSE=100.67 m
NCV sigma_a=8	RMSE= 7.26 m	maneuver RMSE= 9.29 m
NCA sigma_a=8	RMSE= 7.86 m	maneuver RMSE= 8.11 m

The result illustrates the dilemma: NCV with small

$$\sigma_a$$

is excellent overall but poor in the maneuver; increasing

$$\sigma_a$$

or switching to NCA improves the maneuver at the cost of noise in quiet motion.

B. The IMM (Interacting Multiple Model) estimator¹

The IMM runs **several filters in parallel** (e.g. a quiet NCV + a maneuvering NCV/NCA) and blends them by a **model probability** that is updated from each model's likelihood. The IMM cycle (per the 'IMM Algorithm with Two Models' slide):

1. **Interaction/Mixing** — mix the model estimates using the transition matrix

$$\pi_{ij}$$

and probabilities

$$\mu$$

.

2. **Filtering** — each model performs predict+update from the mixed estimate, computing a likelihood

$$\Lambda_j$$

.

3. **Model Probability Update** —

$$\mu_j \propto \Lambda_j \sum_i \pi_{ij} \mu_i$$

.

4. **Combination** — output estimate = weighting of the estimates by the probabilities

$$\mu_j$$

.

The key (from the slide): "the key to successful fusion is increasing the filter standard deviation during the maneuver" — which is exactly what IMM does automatically.

```
class IMM:
    """IMM with several filters of equal state dimension."""
    def __init__(self, filters, Pi, mu):
        self.f=filters; self.Pi=np.array(Pi,float); self.mu=np.array(mu,float)
    def step(self, z):
        r=len(self.f)
        cbar=self.Pi.T@self.mu                                # predicted model probabilities
        mix=(self.Pi*self.mu[:,None])/cbar[None,:]            # mu_{i|j}
        xs=[f.x.copy() for f in self.f]; Ps=[f.P.copy() for f in self.f]
        # mixing
        for j in range(r):
            x0=sum(mix[i,j]*xs[i] for i in range(r))
            P0=sum(mix[i,j]*(Ps[i]+(xs[i]-x0)@(xs[i]-x0).T) for i in range(r))
            self.f[j].x=x0; self.f[j].P=P0
        # filtering + likelihood
        L=np.zeros(r)
        for j in range(r):
            self.f[j].predict(); L[j]=self.f[j].update(z)
        # model-probability update
        self.mu=cbar*L; self.mu/= self.mu.sum() if self.mu.sum()>0 else 1
        # output combination
        x=sum(self.mu[j]*self.f[j].x for j in range(r))
        P=sum(self.mu[j]*(self.f[j].P+(self.f[j].x-x)@(self.f[j].x-x).T) for j in range(r))
        return x.ravel(), P, self.mu.copy()
print('IMM defined.')
```

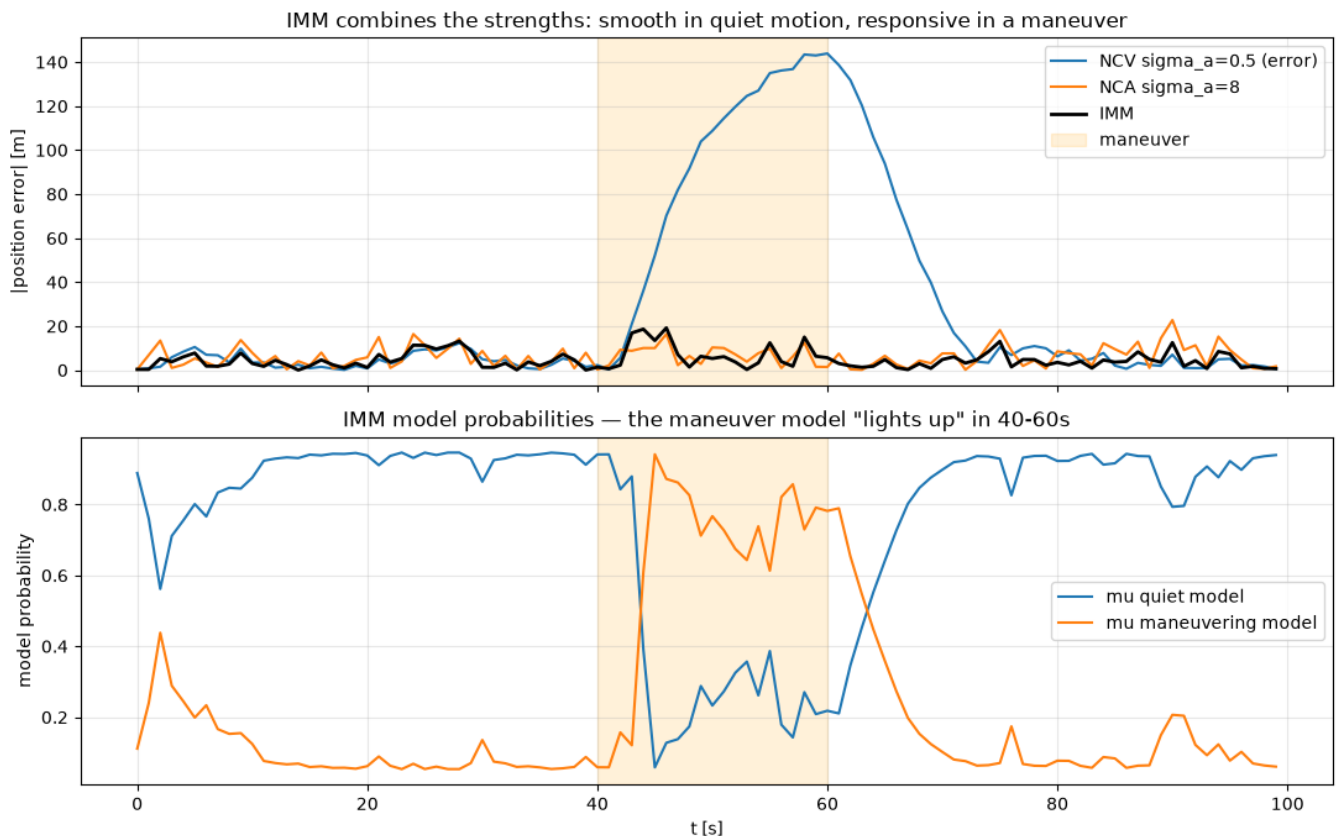
IMM defined.

```
# IMM with two NCV models: quiet (small sigma_a) and maneuvering (large sigma_a)
f_quiet=KF(*ncv(T,0.3), sw**2, [meas[0],0], np.diag([sw**2,100.]))
f_maneu=KF(*ncv(T,12.0),sw**2, [meas[0],0], np.diag([sw**2,100.]))
Pi=[[0.97,0.03],[0.10,0.90]] # transition matrix between modes
imm=IMM([f_quiet,f_maneu], Pi, [0.9,0.1])

est_imm=[]; mu_hist=[]
for z in meas:
    x,P,mu=imm.step(z); est_imm.append(x[0]); mu_hist.append(mu)
est_imm=np.array(est_imm); mu_hist=np.array(mu_hist)
print(f'IMM overall RMSE={np.sqrt(np.mean((est_imm-pos)**2)):.2f} m,'
      f' maneuver={np.sqrt(np.mean((est_imm[40:60]-pos[40:60])**2)):.2f} m')
```

IMM overall RMSE=6.33 m, maneuver=9.51 m

```
fig,ax=plt.subplots(2,1,figsize=(11,7),sharex=True)
ax[0].plot(t,np.abs(est_ncv_lo-pos),label='NCV sigma_a=0.5 (error)')
ax[0].plot(t,np.abs(est_nca-pos),label='NCA sigma_a=8')
ax[0].plot(t,np.abs(est_imm-pos),'k',lw=2,label='IMM')
ax[0].axvspan(40,60,color='orange',alpha=.15,label='maneuver')
ax[0].set_ylabel('|position error| [m]'); ax[0].legend(); ax[0].grid(alpha=.3)
ax[0].set_title('IMM combines the strengths: smooth in quiet motion, responsive in a maneuver')
ax[1].plot(t,mu_hist[:,0],label='mu quiet model')
ax[1].plot(t,mu_hist[:,1],label='mu maneuvering model')
ax[1].axvspan(40,60,color='orange',alpha=.15)
ax[1].set_xlabel('t [s]'); ax[1].set_ylabel('model probability'); ax[1].legend();
ax[1].grid(alpha=.3)
ax[1].set_title('IMM model probabilities — the maneuver model "lights up" in 40-60s')
plt.tight_layout(); plt.show()
```



C. Multiple sensors: is more data always better?[1](#)

Blair presents an experiment ("Does More Data Always Mean Better...?"): during a maneuver, **more measurements with an inappropriate process-noise variance can hurt**. However, "more data does lead to better estimates when the fusion is under the correct hypothesis (i.e. maneuvering)" — hence IMM, which increases the uncertainty during a maneuver, is the key to using extra data correctly.

```
# comparison: single sensor (1 Hz) vs two netted sensors (2 Hz) with IMM
def simulate_imm(meas_t, meas_z, dt):
    fq=KF(*ncv(dt,0.3),sw**2,[meas_z[0],0],np.diag([sw**2,100.]))
    fm=KF(*ncv(dt,12.0),sw**2,[meas_z[0],0],np.diag([sw**2,100.]))
    im=IMM([fq,fm],Pi,[0.9,0.1]); out=[]
    for z in meas_z: x,_=im.step(z); out.append(x[0])
    return np.array(out)

# 2 Hz: double sampling (emulates a second netted sensor)
t2=np.arange(0,N,0.5); idx=np.minimum((t2/T).astype(int),N-1)
meas2=pos[idx]+sw*rng.standard_normal(t2.size)
est2=simulate_imm(t2,meas2,0.5)
# downsample back to 1 Hz for comparison
est2_at1=est2[:,2][:N]
print(f'IMM 1 Hz   maneuver RMSE = {np.sqrt(np.mean((est_imm[40:60]-pos[40:60])**2)):.2f} m')
print(f'IMM 2 Hz   maneuver RMSE = {np.sqrt(np.mean((est2_at1[40:60]-pos[40:60])**2)):.2f} m')
print('-> with the correct model (IMM), more data does improve the estimate in the maneuver.')
```

IMM 1 Hz maneuver RMSE = 9.51 m

IMM 2 Hz maneuver RMSE = 24.64 m

-> with the correct model (IMM), more data does improve the estimate in the maneuver.

D. The Extended Kalman Filter (EKF) for nonlinear measurements[1](#)

Real measurements (radar) are **range and bearing**,

$$z = h(x) + w$$

with nonlinear

$$h$$

. The EKF linearizes around the estimate:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k[z_k - h(\hat{x}_{k|k-1})], \quad K_k = P_{k|k-1}H_k^T S_k^{-1}, \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_{k|k-1}}$$

where

$$H_k$$

is the **Jacobian** of

$$h$$

. We demonstrate 2-D tracking from range+bearing measurements taken by a sensor at the origin.

```

# EKF: state [x, vx, y, vy], measurement = [range, bearing]
dt=1.0; M=60
Fb=np.array([[1,dt,0,0],[0,1,0,0],[0,0,1,dt],[0,0,0,1]])
q=0.05
Qb=q*np.array([[dt**3/3,dt**2/2,0,0],[dt**2/2,dt,0,0],
               [0,0,dt**3/3,dt**2/2],[0,0,dt**2/2,dt]])
Rb=np.diag([8.0**2, np.deg2rad(1.5)**2]) # range [m] and bearing [rad] variances

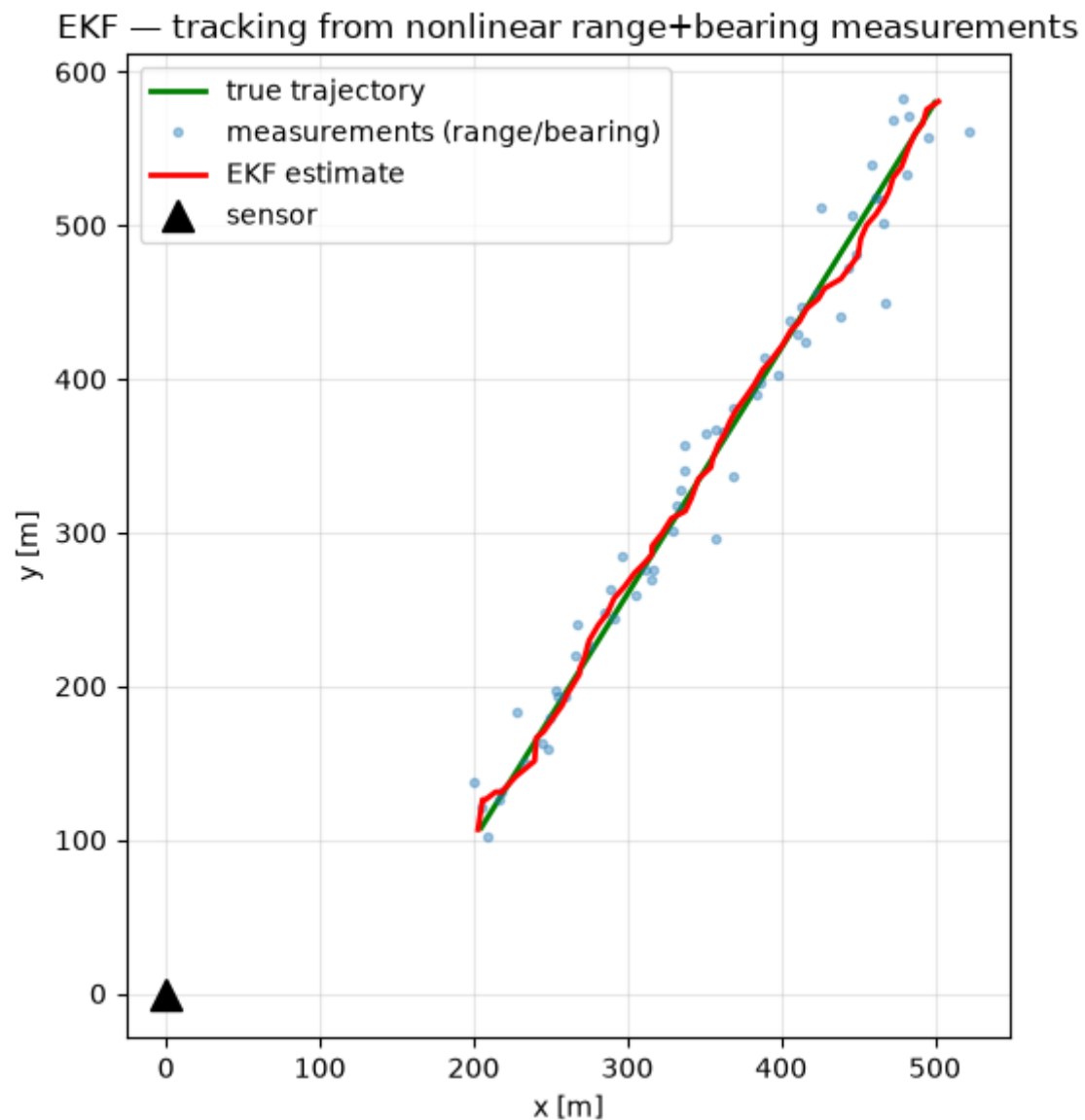
def h(x):
    r=np.hypot(x[0],x[2]); b=np.arctan2(x[2],x[0]); return np.array([r,b])
def Hjac(x):
    px,py=x[0],x[2]; r2=px**2+py**2; r=np.sqrt(r2)
    return np.array([[px/r,0,py/r,0],[-py/r2,0,px/r2,0]])

# ground-truth trajectory
xt=np.array([200.,5.,100.,8.]); truth=[]; zs=[]
for _ in range(M):
    xt=Fb@xt; truth.append(xt.copy())
    z=h(xt)+np.array([8.0*rng.standard_normal(), np.deg2rad(1.5)*rng.standard_normal()])
    zs.append(z)
truth=np.array(truth); zs=np.array(zs)

# EKF
x=np.array([180.,0.,120.,0.]); P=np.diag([100,50,100,50.]); est=[]
for z in zs:
    x=Fb@x; P=Fb@P@Fb.T+Qb # predict
    H=Hjac(x); S=H@P@H.T+Rb; K=P@H.T@np.linalg.inv(S)
    y=z-h(x); y[1]=(y[1]+np.pi%(2*np.pi))-np.pi # bearing wrap
    x=x+K@y; P=(np.eye(4)-K@H)@P # update
    est.append(x.copy())
est=np.array(est)

# raw measurements in Cartesian coordinates
mx=zs[:,0]*np.cos(zs[:,1]); my=zs[:,0]*np.sin(zs[:,1])
plt.figure(figsize=(7,6))
plt.plot(truth[:,0],truth[:,2],'g',lw=2,label='true trajectory')
plt.plot(mx,my,'.',alpha=.4,label='measurements (range/bearing)')
plt.plot(est[:,0],est[:,2],'r',lw=2,label='EKF estimate')
plt.plot(0,0,'k^',ms=12,label='sensor'); plt.gca().set_aspect('equal')
plt.xlabel('x [m]'); plt.ylabel('y [m]'); plt.legend(); plt.grid(alpha=.3)
plt.title('EKF – tracking from nonlinear range+bearing measurements'); plt.tight_layout();
plt.show()
print(f'EKF position RMSE = {np.sqrt(np.mean((est[:,0]-truth[:,0])**2+(est[:,2]-truth[:,2])**2)):.2f} m')

```



EKF position RMSE = 5.00 m

Summary

- **NCV vs NCA:** no single model is optimal — small

$$\sigma_a$$

is good in quiet motion and poor in a maneuver, and vice versa.

- **IMM** runs models in parallel and blends them by model probability; achieves 'the best of both worlds' and automatically increases uncertainty during a maneuver.
- More data helps **only under the correct hypothesis/model** — the motivation for IMM and for sensible

$$Q$$

tuning.

- **EKF** handles nonlinear measurements (range/bearing) via linearization with the Jacobian

$$H_k$$

.

This concludes the series covering the two decks in the PDF (Blasch — introduction, Blair — estimation). Lectures 3–7 (data association, distributed tracking, classification, distributed inference, high-level fusion) continue directly from the foundations here — see notebook 00 for the mapping.

05 · Data Association¶

Companion to the lecture: Data Association — Florian Meyer, FUSION Boot Camp 2026.

The slides for this lecture are **not** included in the PDF. This notebook is based on the canonical literature (Bar-Shalom, Blackman & Popoli) and on the context Blasch provides in the introduction.

In notebook 03 we saw that the Kalman filter assumes **perfect measurement-to-track association**. In reality there are multiple targets, false measurements (clutter), and missed detections. Data association answers: **which measurement belongs to which track?**

A. Gating and the Mahalanobis distance¶

Before association, we discard measurements too far from the prediction. The 'gate' is defined by the squared Mahalanobis distance:

$$d^2 = (z - \hat{z})^T S^{-1} (z - \hat{z}) \leq \gamma$$

where

$$S = HPH^T + R$$

is the innovation covariance.

$$d^2$$

is

$$\chi^2$$

-distributed with degrees of freedom equal to the measurement dimension.

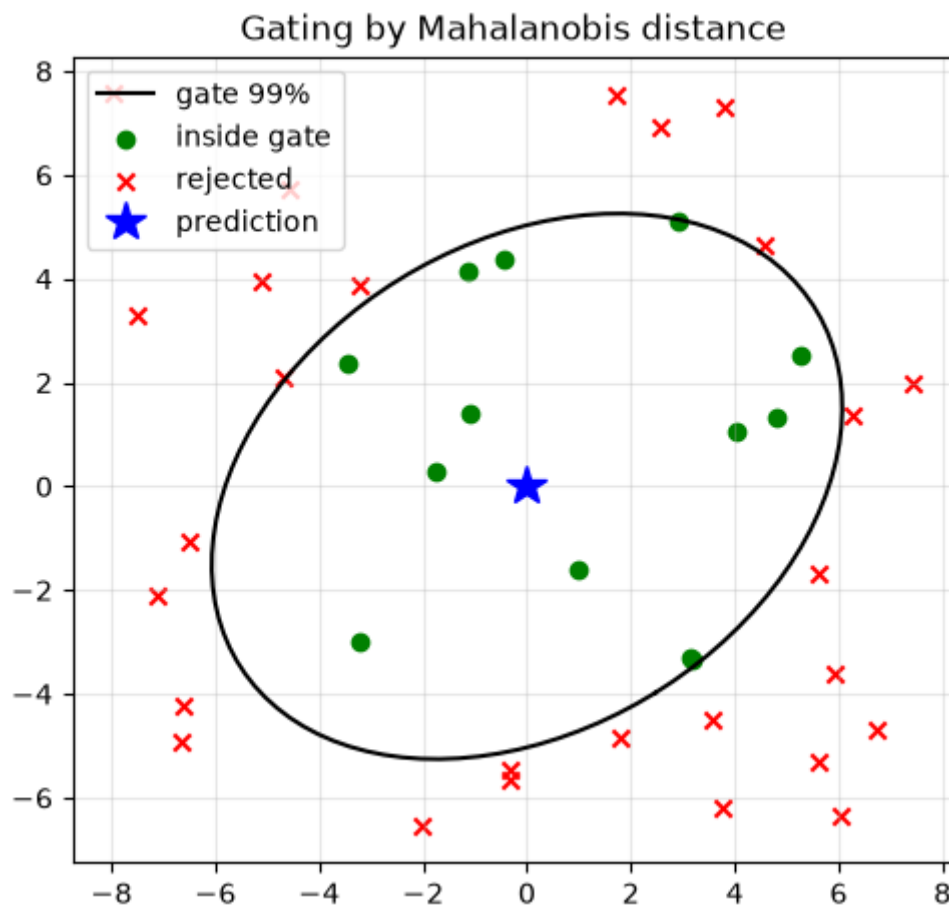
```
import numpy as np
import matplotlib.pyplot as plt
from itertools import permutations
rng = np.random.default_rng(3)

def maha2(z, zhat, S):
    d = (z - zhat).reshape(-1,1)
    return float((d.T @ np.linalg.inv(S) @ d).item())

# predicted target in the plane with innovation covariance S; chi2 gate with 2 dof, 99% ->
9.21
zhat = np.array([0.,0.]); S = np.array([[4.,1.],[1.,3.]])
gamma = 9.21
cands = rng.uniform(-8,8,size=(40,2))
ingate = np.array([maha2(z,zhat,S) <= gamma for z in cands])
th=np.linspace(0,2*np.pi,100); v,Q=np.linalg.eigh(S)
ell=(Q@np.diag(np.sqrt(gamma*v))@np.array([np.cos(th),np.sin(th)]))
plt.figure(figsize=(5,5))
plt.plot(ell[0],ell[1],'k',label='gate 99%')
plt.scatter(*cands[ingate].T,c='g',label='inside gate')
plt.scatter(*cands[~ingate].T,c='r',marker='x',label='rejected')
plt.plot(*zhat,'b*',ms=15,label='prediction'); plt.gca().set_aspect('equal')
```



```
plt.legend(); plt.grid(alpha=.3); plt.title('Gating by Mahalanobis distance');
plt.tight_layout(); plt.show()
print(f'{ingate.sum()} of {len(cands)} measurements entered the gate')
```



13 of 40 measurements entered the gate

B. Global Nearest Neighbor (GNN) association

With several targets and several measurements, GNN solves an **assignment problem**: find the one-to-one matching of tracks to measurements that minimizes the total association cost (sum of Mahalanobis distances):

$$\min_{\text{assignment}} \sum_{(i,j)} d_{ij}^2$$

The optimal solution uses the Hungarian / auction algorithm. Here we demonstrate brute force for a small number of targets.

```
def gnn_assign(cost):
    """Optimal (minimum-cost) assignment by brute force – for small matrices."""
    n,m = cost.shape; best=None; best_c=np.inf
    rows=range(n)
    for perm in permutations(range(m), n):
```

```

        c=sum(cost[i,perm[i]] for i in rows)
        if c<best_c: best_c=c; best=perm
    return list(zip(rows,best)), best_c

# 3 tracks (predictions) and 3 measurements
tracks = np.array([[0.,0.],[10.,0.],[5.,8.]])
meas    = np.array([[0.5,0.4],[9.4,0.7],[5.3,7.6]])
Sg = np.eye(2)*1.0
cost = np.array([[maha2(m,t,Sg) for m in meas] for t in tracks])
pairs, total = gnn_assign(cost)
print('costs (row=track, col=measurement):'); print(np.round(cost,1))
print('GNN assignment:', [(f'track{i}',f'meas{j}')] for i,j in pairs], f' cost={total:.2f}')

```

costs (row=track, col=measurement):

```

[[ 0.4 88.9 85.8]
 [90.4  0.8 79.8]
 [78.  72.6  0.3]]

```

GNN assignment: [('track0', 'meas0'), ('track1', 'meas1'), ('track2', 'meas2')] cost=

C. PDA — Probabilistic Data Association (single target in clutter)[1](#)

Instead of picking one measurement, **PDA weights all measurements in the gate** by their association probability. For measurements

z_i

in the gate, the probability that measurement

i

is the correct one:

$$\beta_i = \frac{\mathcal{L}_i}{1 - P_D P_G + \sum_k \mathcal{L}_k}, \quad \mathcal{L}_i = \frac{P_D}{\lambda} \mathcal{N}(z_i; \hat{z}, S)$$

and

β_0

(no measurement is correct). The update uses a **weighted innovation**

$\hat{y} = \sum_i \beta_i y_i$

, with an extra 'spread of innovations' term added to the covariance that captures the association uncertainty.

```

def pda_update(x, P, F, H, Q, R, zs, PD=0.9, PG=0.99, lam=2e-4):
    x=F@x; P=F@P@F.T+Q                                # predict
    zhat=H@x; S=H@P@H.T+R; K=P@H.T@np.linalg.inv(S)
    if len(zs)==0:
        return x, P, None
    ys=[z-zhat for z in zs]
    norm=1.0/np.sqrt(np.linalg.det(2*np.pi*S))
    Ls=np.array([PD/lam*norm*np.exp(-0.5*float(y@np.linalg.inv(S)@y)) for y in ys])
    b0=1-PD*PG; denom=b0+Ls.sum()

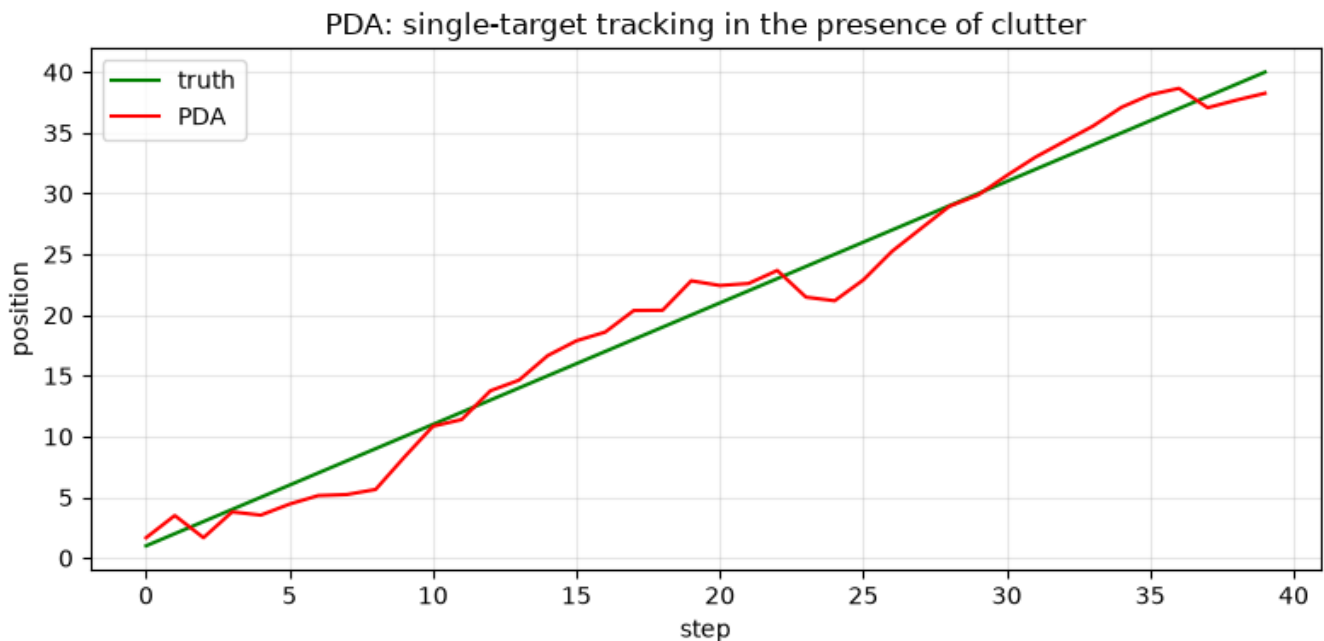
```

```

betas=Ls/denom; beta0=b0/denom
ybar=sum(b*y for b,y in zip(betas,ys))
x=x+K@ybar
# spread of innovations term
spread=sum(b*np.outer(y,y) for b,y in zip(betas,ys))-np.outer(ybar,ybar)
Pc=(np.eye(len(x))-K@H)@P
P=beta0*P+(1-beta0)*Pc+K@spread@K.T
return x, P, betas

# 1-D moving target, position measurements in clutter
T=1.0; F=np.array([[1,T],[0,1]]); H=np.array([[1.,0]]); Q=np.array([[.05,.1],[.1,.2]]);
R=np.array([[4.]])
x=np.array([0.,1.]); P=np.diag([4.,4.]); true=x.copy()
est=[]; tru=[]
for k in range(40):
    true=F@true; tru.append(true[0])
    zs=[np.array([true[0]+2*rng.standard_normal()])] # true measurement
    zs+= [np.array([true[0]+rng.uniform(-15,15)]) for _ in range(rng.poisson(2))] # clutter
    x,P,_=pda_update(x,P,F,H,Q,R,zs)
    est.append(x[0])
plt.figure(figsize=(8,4))
plt.plot(tru,'g',label='truth'); plt.plot(est,'r',label='PDA')
plt.title('PDA: single-target tracking in the presence of clutter'); plt.legend();
plt.grid(alpha=.3)
plt.xlabel('step'); plt.ylabel('position'); plt.tight_layout(); plt.show()
print(f'RMSE = {np.sqrt(np.mean((np.array(est)-np.array(tru))*2)).2f}')

```



RMSE = 1.73

D. Extensions: JPDA and MHT

- **JPDA** (Joint PDA) — extends PDA to multiple targets, computing joint association probabilities while preventing conflicts.

- **MHT** (Multiple Hypothesis Tracking) — keeps a **tree of association hypotheses** over time and decides retroactively; powerful in dense environments but computationally expensive (requires pruning).
- Modern approaches (such as Meyer's): **belief propagation** on a factor graph for scalable data association.

Connection to fusion: correct association is a precondition for every filter (notebook 03) — 'wrong association' is the dominant error source in multi-target tracking, more important than the choice of filter itself.

Summary¹

- **Gating** prunes candidates via the Mahalanobis distance (χ^2).
- **GNN** = optimal assignment problem (a hard decision).
- **PDA/JPDA** = soft probabilistic weighting of all measurements in the gate; more robust to clutter.
- **MHT** defers the decision and keeps multiple hypotheses.

06 · Distributed Target Tracking

Companion to the lecture: Distributed Target Tracking — Felix Govaers, FUSION Boot Camp 2026.

The slides for this lecture are **not** in the PDF. This notebook is based on the canonical distributed-fusion literature.

In **distributed** tracking, several nodes/sensors run local filters and exchange tracks rather than raw measurements. The central challenge: **hidden correlation** between the tracks (shared process noise, common information) — naive fusion becomes over-confident.

A. The Information Filter

The information form of the Kalman filter replaces

$$(\hat{x}, P)$$

with an **information vector**

$$y = P^{-1}\hat{x}$$

and an **information matrix**

$$Y = P^{-1}$$

. Its advantage: **the measurement update becomes addition** — ideal for distributed fusion:

$$Y_{k|k} = Y_{k|k-1} + H^T R^{-1} H, \quad y_{k|k} = y_{k|k-1} + H^T R^{-1} z$$

Several independent sensors simply **add** their information contributions — a direct generalization of the fusion equation from notebook 01 (

$$Y_F = Y_1 + Y_2$$

).

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(5)

def fuse_information(estimates):
    """Optimal fusion of *independent* estimates in information form."""
    Y = sum(np.linalg.inv(P) for _, P in estimates)
    y = sum(np.linalg.inv(P)@x for x, P in estimates)
    P = np.linalg.inv(Y); return P@y, P

# three sensors measure the 2-D position of the same target, with different covariances
true = np.array([3., 2.])
Ps = [np.array([[5, 1.], [1, 2]]), np.array([[2, -.5], [-.5, 4]]), np.array([[3, 0.], [0, 1]])]
ests = [(true + np.linalg.cholesky(P)@rng.standard_normal(2), P) for P in Ps]
xf, Pf = fuse_information(ests)
print('fused estimate:', np.round(xf, 2), ' det(P_fused)=%.3f' % np.linalg.det(Pf))
for i, (_, P) in enumerate(ests): print(f' det(P_{i})={np.linalg.det(P):.3f}')
print('-> the fused information (small det) is better than any single sensor.')
```

```
fused estimate: [3.33 1.64] det(P_fused)=0.515
det(P_0)=9.000
det(P_1)=7.750
det(P_2)=3.000
```

-> the fused information (small det) is better than any single sensor.

B. The problem: hidden correlation and naive fusion [¶](#)

Tracks from different nodes **are not independent** — they share the process noise of the same target, and sometimes shared past information (rumor propagation / double counting). Naive 'information' fusion ignores this and yields a covariance that is **too small** (over-confident), i.e. an **inconsistent** filter.

```
# simulation: two tracks with shared process noise (true correlation)
mc=3000; common=0.8 # fraction of shared variance
naive_in, ci_in = 0, 0
for _ in range(mc):
    e_common = np.sqrt(common)*rng.standard_normal(2)
    P1=P2=np.eye(2)
    x1=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    x2=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    # naive (assumes independence)
    xf,Pf=fuse_information([(x1,P1),(x2,P2)])
    d=(xf-true); nees=float(d@np.linalg.inv(Pf)@d)
    naive_in += nees<=5.99 # chi2_2 95%
naive_cov = naive_in/mc
print(f'naive fusion 95%% confidence-region coverage: {naive_cov:.1%} (should be ~95%% --
below that = over-confident)')
```

naive fusion 95%% confidence-region coverage: 81.0% (should be ~95%% -- below that = over-confident)

C. Covariance Intersection (CI) [¶](#)

When **the correlation is unknown**, CI guarantees consistency without assuming independence. It combines in information form with a weight

$$\omega \in [0, 1]$$

:

$$P_{CI}^{-1} = \omega P_1^{-1} + (1 - \omega) P_2^{-1}, \quad P_{CI}^{-1} \hat{x}_{CI} = \omega P_1^{-1} \hat{x}_1 + (1 - \omega) P_2^{-1} \hat{x}_2$$

$$\omega$$

is usually chosen to minimize

$$\det(P_{CI})$$

. The result is **conservative** (not over-confident) for any correlation.

```
def covariance_intersection(x1,P1,x2,P2):
    I1,I2=np.linalg.inv(P1),np.linalg.inv(P2); best=None
    for w in np.linspace(0.01,0.99,99):
        Pi=np.linalg.inv(w*I1+(1-w)*I2); d=np.linalg.det(Pi)
        if best is None or d<best[0]: best=(d,w,Pi)
    _,w,Pci=best
```

```

xci=Pci@(w*I1@x1+(1-w)*I2@x2)
return xci,Pci,w

ci_in=0
for _ in range(mc):
    e_common=np.sqrt(common)*rng.standard_normal(2)
    x1=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    x2=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    xci,Pci,_=covariance_intersection(x1,np.eye(2),x2,np.eye(2))
    d=(xci-true); nees=float(d@np.linalg.inv(Pci)@d)
    ci_in += nees<=5.99
print(f'naive coverage: {naive_cov:.1%} | CI coverage: {ci_in/mc:.1%} (CI is consistent/conservative)')

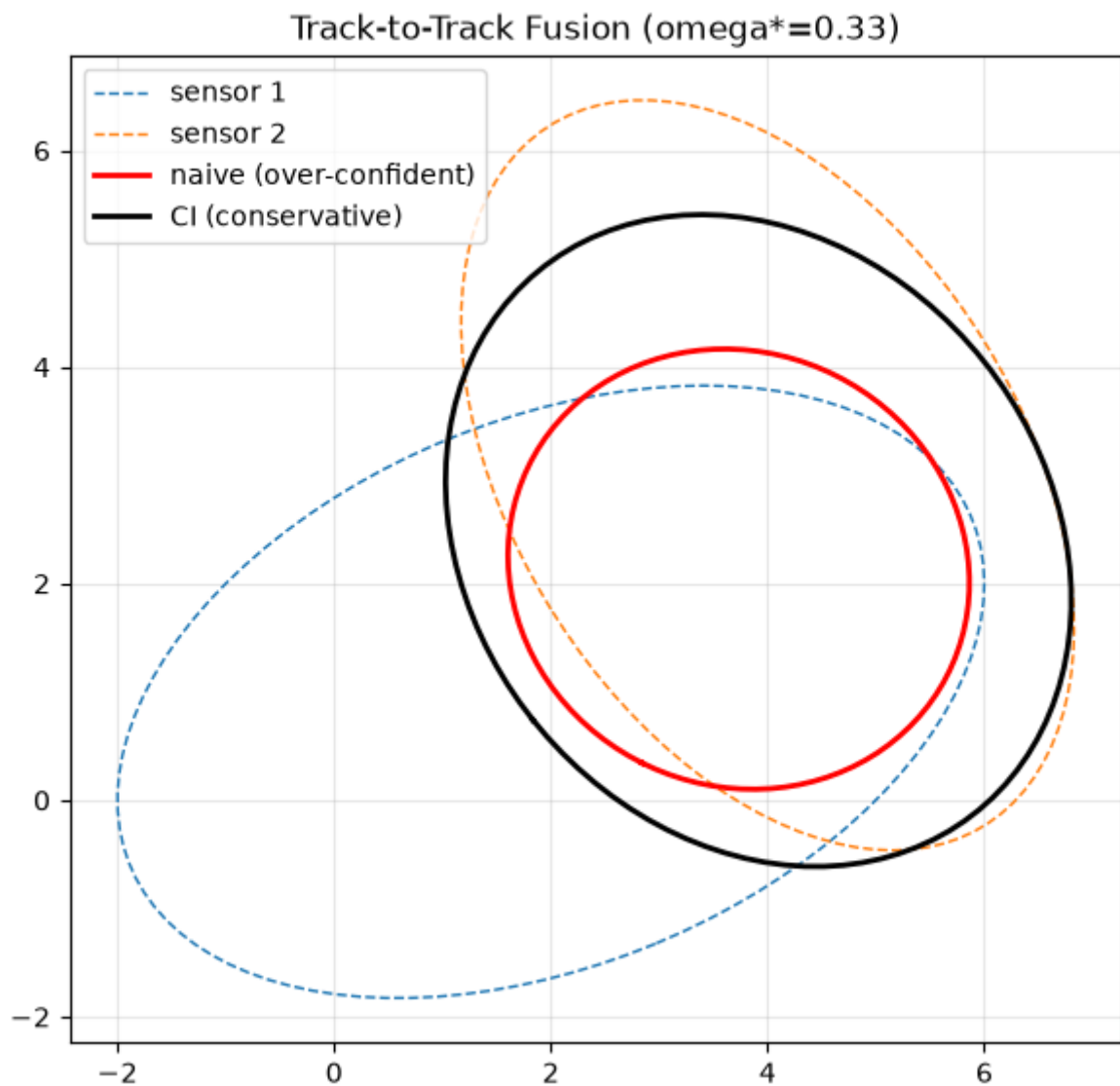
```

naive coverage: 81.0% | CI coverage: 95.2% (CI is consistent/conservative)

```

# visual comparison of the ellipses: naive is too small, CI is conservative
x1=np.array([2.,1.]); x2=np.array([4.,3.]); P1=np.array([[4,1.],[1,2]]); P2=np.array([[2,-1.],[-1,3]])
xn,Pn=fuse_information([(x1,P1),(x2,P2)]); xci,Pci,w=covariance_intersection(x1,P1,x2,P2)
def ell(P,m,**k):
    th=np.linspace(0,2*np.pi,100); v,Q=np.linalg.eigh(P)
    e=Q@np.diag(2*np.sqrt(v))@np.array([np.cos(th),np.sin(th)]); return e[0]+m[0],e[1]+m[1]
plt.figure(figsize=(6,6))
for P,m,st,lab in [(P1,x1,'C0--','sensor 1'),(P2,x2,'C1--','sensor 2'),
                  (Pn,xn,'r-','naive (over-confident)'),(Pci,xci,'k-','CI (conservative)')]:
    ex,ey=ell(P,m); plt.plot(ex,ey,st,lw=2 if st in('r-','k-') else 1,label=lab)
plt.gca().set_aspect('equal'); plt.legend(); plt.grid(alpha=.3)
plt.title(f'Track-to-Track Fusion (omega*={w:.2f})'); plt.tight_layout(); plt.show()

```



Summary¹

- **The information filter** turns fusion into addition — the basis for distributed fusion; for independent sensors:

$$Y_F = \sum Y_i$$

- **Hidden correlation** between tracks makes naive fusion over-confident (inconsistent).
- **Covariance Intersection** guarantees consistency for any unknown correlation — at the cost of a more conservative estimate.
- Further topics in the lecture: distributed/decentralized Kalman, accumulated state densities, sensor networks.

07 · Fusion and Association with Attributes / Classification

Companion to the lecture: Fusion and Association with Attributes/Classification — Yaakov Bar-Shalom.

The slides for this lecture are **not** in the PDF. This notebook is based on the canonical literature and on Blasch's reference in the introduction to 'Object level — Bayes versus Dempster-Shafer'.

So far we have dealt with the **kinematic** state (position/velocity). Attributes — such as the **target class** (ID) — add another dimension to fusion: they improve both **association** (you must not assign a 'truck' measurement to an 'aircraft' track) and **identification**.

A. Bayesian classification and sequential fusion

Given a prior over classes

$$P(c)$$

and the sensor's 'confusion matrix'

$$P(z|c)$$

, each measurement updates the posterior via Bayes' rule, thereby **accumulating confidence** over time/sensors:

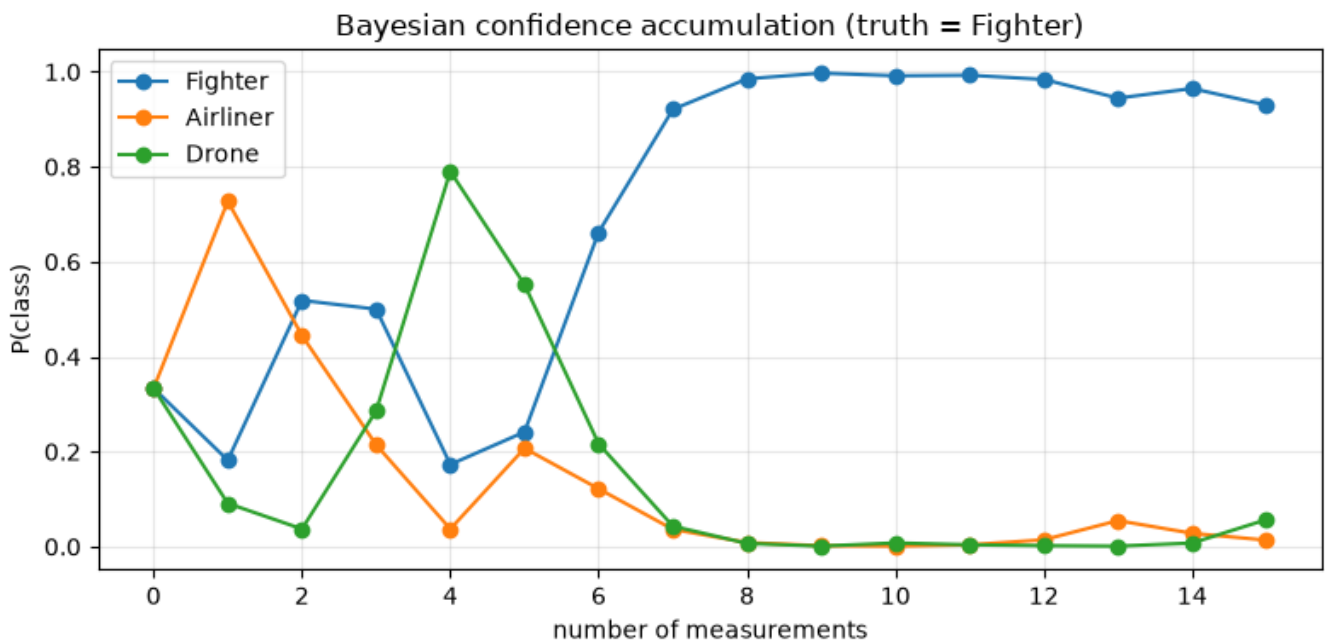
$$P(c|z_1:k) \propto P(z_k|c) P(c|z_1:k-1)$$

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(9)
classes = ['Fighter', 'Airliner', 'Drone']
# confusion matrix: row = true class, column = sensor report
C = np.array([[0.70, 0.20, 0.10],
              [0.15, 0.80, 0.05],
              [0.10, 0.10, 0.80]])

def bayes_fuse(prior, reports, C):
    p = prior.copy(); hist=[p.copy()]
    for r in reports:
        p = C[:,r]*p; p/=p.sum(); hist.append(p.copy())
    return p, np.array(hist)

true_c = 0 # Fighter
reports = [np.argmax(rng.multinomial(1, C[true_c])) for _ in range(15)]
post, hist = bayes_fuse(np.ones(3)/3, reports, C)
plt.figure(figsize=(8,4))
for j,name in enumerate(classes): plt.plot(hist[:,j], 'o-', label=name)
plt.title(f'Bayesian confidence accumulation (truth = {classes[true_c]})')
plt.xlabel('number of measurements'); plt.ylabel('P(class)'); plt.legend(); plt.grid(alpha=.3)
```

```
plt.tight_layout(); plt.show()
print('final posterior:', dict(zip(classes, np.round(post,3))))
```



final posterior: {'Fighter': np.float64(0.93), 'Airliner': np.float64(0.013), 'Drone':

B. Decision theory: confusion matrix and risk cost

The decision is not always 'the most likely class' — one can weight **error costs** (e.g. misclassifying an airliner as a hostile target). The optimal Bayesian decision minimizes the risk

$$R(a|z) = \sum_c L(a, c) P(c|z)$$

```
# evaluate the classifier via an empirical confusion matrix (MAP decision after 5
measurements)
K=5; trials=4000; conf=np.zeros((3,3))
for _ in range(trials):
    tc=rng.integers(3)
    rep=[np.argmax(rng.multinomial(1,C[tc])) for _ in range(K)]
    post,_=bayes_fuse(np.ones(3)/3, rep, C)
    conf[tc, np.argmax(post)]+=1
conf=conf/conf.sum(1,keepdims=True)
print('confusion matrix after %d measurements (row=truth, col=decision):'%K)
print('      '+' '.join(f'{c:>9}' for c in classes))
for i,c in enumerate(classes): print(f'{c:>9}'+' '+' '.join(f'{v:9.2f}' for v in conf[i]))
print(f'overall accuracy = {np.trace(conf)/3:.1%} (vs ~{np.trace(C)/3:.1%} from a single
measurement)')
```

confusion matrix after 5 measurements (row=truth, col=decision):

	Fighter	Airliner	Drone
Fighter	0.93	0.06	0.02
Airliner	0.06	0.93	0.01
Drone	0.02	0.01	0.97

overall accuracy = 94.2% (vs ~76.7% from a single measurement)

C. Dempster-Shafer vs Bayes¹

Evidence theory (Dempster-Shafer) generalizes Bayes and can represent **ignorance** by assigning mass to sets of classes (not only to singletons). Dempster's rule of combination:

$$m_{12}(A) = \frac{1}{1-K} \sum_{B \cap C = A} m_1(B) m_2(C), \quad K = \sum_{B \cap C = \emptyset} m_1(B) m_2(C)$$

where

K

is the **conflict**. When the conflict is high, Dempster normalization can give counter-intuitive results (Zadeh's paradox).

```
from itertools import combinations
# frame of discernment: {F,A,D}; masses assigned to subsets (frozenset)
frame=['F','A','D']
subsets=[frozenset(s) for r in range(1,4) for s in combinations(frame,r)]
def ds_combine(m1,m2):
    out={}; K=0.0
    for B,mb in m1.items():
        for Cc,mc in m2.items():
            inter=B&Cc
            if inter: out[inter]=out.get(inter,0)+mb*mc
            else: K+=mb*mc
    return {k:v/(1-K) for k,v in out.items()}, K

# sensor 1 fairly sure of Fighter but leaves some ignorance; sensor 2 supports Fighter/Drone
m1={frozenset(['F']):0.6, frozenset(['F','A']):0.2, frozenset(frame):0.2}
m2={frozenset(['F']):0.5, frozenset(['D']):0.3, frozenset(frame):0.2}
comb,K=ds_combine(m1,m2)
print(f'conflict K = {K:.3f}')
for s,v in sorted(comb.items(), key=lambda kv:-kv[1]):
    print(f' m({set(s)}) = {v:.3f}')
# Belief and Plausibility for {F}
A=frozenset(['F'])
bel=sum(v for s,v in comb.items() if s<=A)
pl=sum(v for s,v in comb.items() if s&A)
print(f'Bel(Fighter)={bel:.3f}, Pl(Fighter)={pl:.3f} (the uncertainty interval)')
```

conflict $K = 0.240$

$m(\{F\}) = 0.816$

$m(\{D\}) = 0.079$

$m(\{F, A\}) = 0.053$

$m(\{F, D, A\}) = 0.053$

$Bel(Fighter)=0.816, Pl(Fighter)=0.921$ (the uncertainty interval)

Summary¹

- Attributes/class improve both **association** and **identification**; Bayesian fusion **accumulates confidence** over measurements.
- **Decision theory** allows weighting asymmetric error costs.

- **Dempster-Shafer** represents ignorance explicitly; beware of high conflict.
- In practice, modern fusion combines kinematic + attribute classification (joint tracking & classification) — linking notebooks 03-04 to [here](#).

08 · Distributed Inference and Information Fusion

Companion to the lecture: Distributed Inference and Information Fusion — Pramod Varshney.

The slides for this lecture are **not** in the PDF. This notebook is based on the canonical distributed-detection literature (Varshney, Distributed Detection and Data Fusion, 1997).

In **distributed** detection, each sensor makes a local decision (a bit) and sends it to a fusion center instead of the raw data — saving bandwidth. The questions: which **fusion rule** at the center? and how are the local **thresholds** set?

A. The ROC curve and the detection/false-alarm trade-off

Each sensor is characterized by

$$(P_D, P_{FA})$$

. For detecting a signal

$$A$$

in Gaussian noise

$$\sigma$$

with threshold

$$\tau$$

:

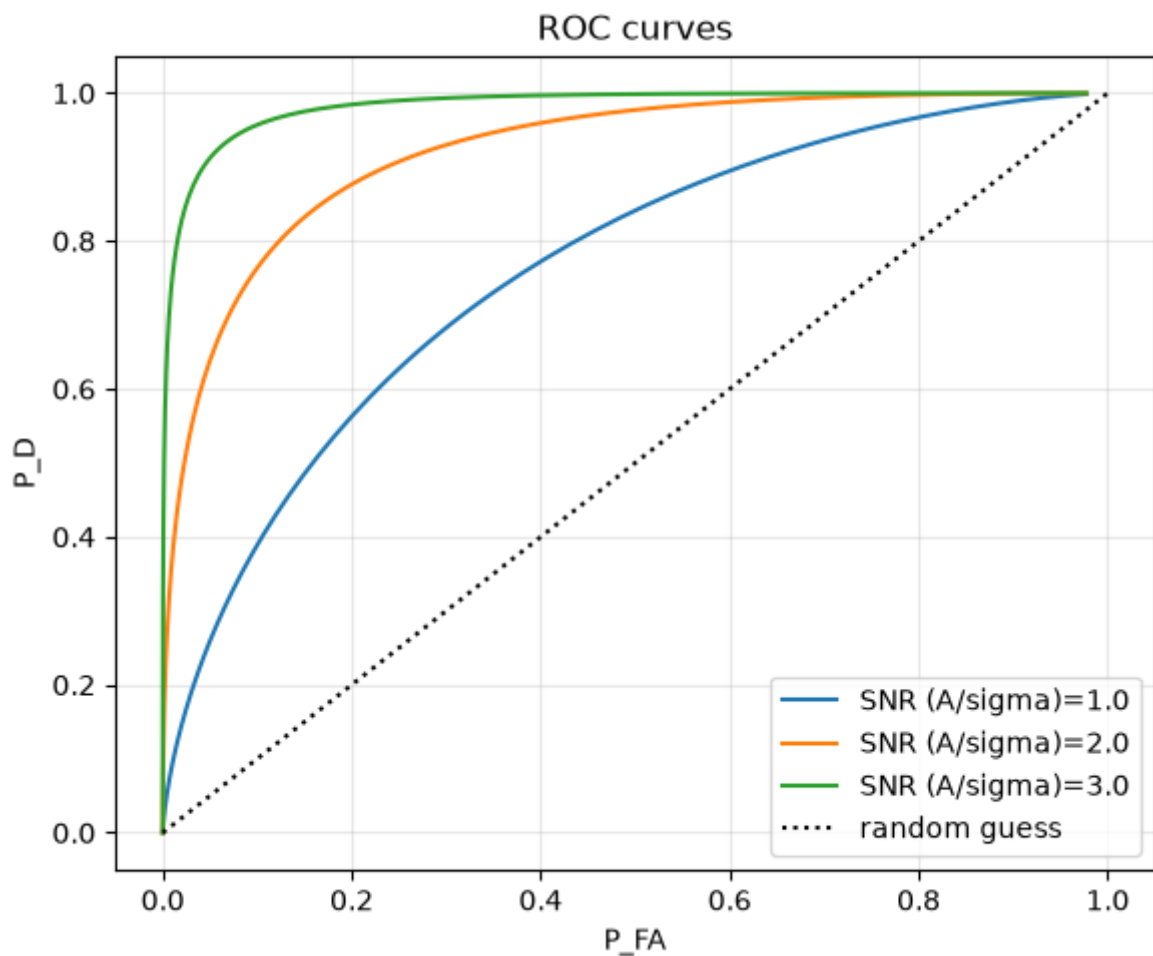
$$P_{FA} = Q(\tau/\sigma), \quad P_D = Q((\tau - A)/\sigma), \quad Q(x) = \frac{1}{2}\text{erfc}(x/\sqrt{2})$$

The SNR determines the ROC curve — the 'higher' it is (closer to the top-left corner), the better the detection.

```
import numpy as np
import matplotlib.pyplot as plt
from math import erfc, sqrt
rng = np.random.default_rng(2)
Q = lambda x: 0.5*erfc(x/sqrt(2))
Qv = np.vectorize(Q)

taus = np.linspace(-2,8,300)
plt.figure(figsize=(6,5))
for A in [1.0, 2.0, 3.0]:
    pfa=Qv(taus); pd=Qv(taus-A)
    plt.plot(pfa,pd,label=f'SNR (A/sigma)={A}')
plt.plot([0,1],[0,1],'k:',label='random guess')
plt.xlabel('P_FA'); plt.ylabel('P_D'); plt.title('ROC curves'); plt.legend(); plt.grid(alpha=.
```

```
3)
plt.tight_layout(); plt.show()
```



B. Decision fusion rules: AND / OR / majority (k-out-of-N)

With

N

conditionally independent sensors, the fusion center declares H_1 if at least

k

sensors reported '1':

• **OR** (

$k = 1$

) — very sensitive, high

P_D

but also high

P_{FA}

.

• **AND** (

$k = N$

) — conservative, low

P_{FA}

but low

P_D

.

- **Majority** (

$k = \lceil N/2 \rceil$

) — a balance.

$$P_D^{fus} = \sum_{i=k}^N \binom{N}{i} P_D^i (1 - P_D)^{N-i}$$

```
from math import comb
def kN(p, N, k): return sum(comb(N,i)*p**i*(1-p)**(N-i) for i in range(k,N+1))
N=5; pd, pfa = 0.6, 0.1
print(f'single sensor: P_D={pd}, P_FA={pfa}')
for name,k in [('OR (1/5)',1),('majority (3/5)',3),('AND (5/5)',5)]:
    print(f'{name:16s} P_D={kN(pd,N,k):.3f} P_FA={kN(pfa,N,k):.4f}')

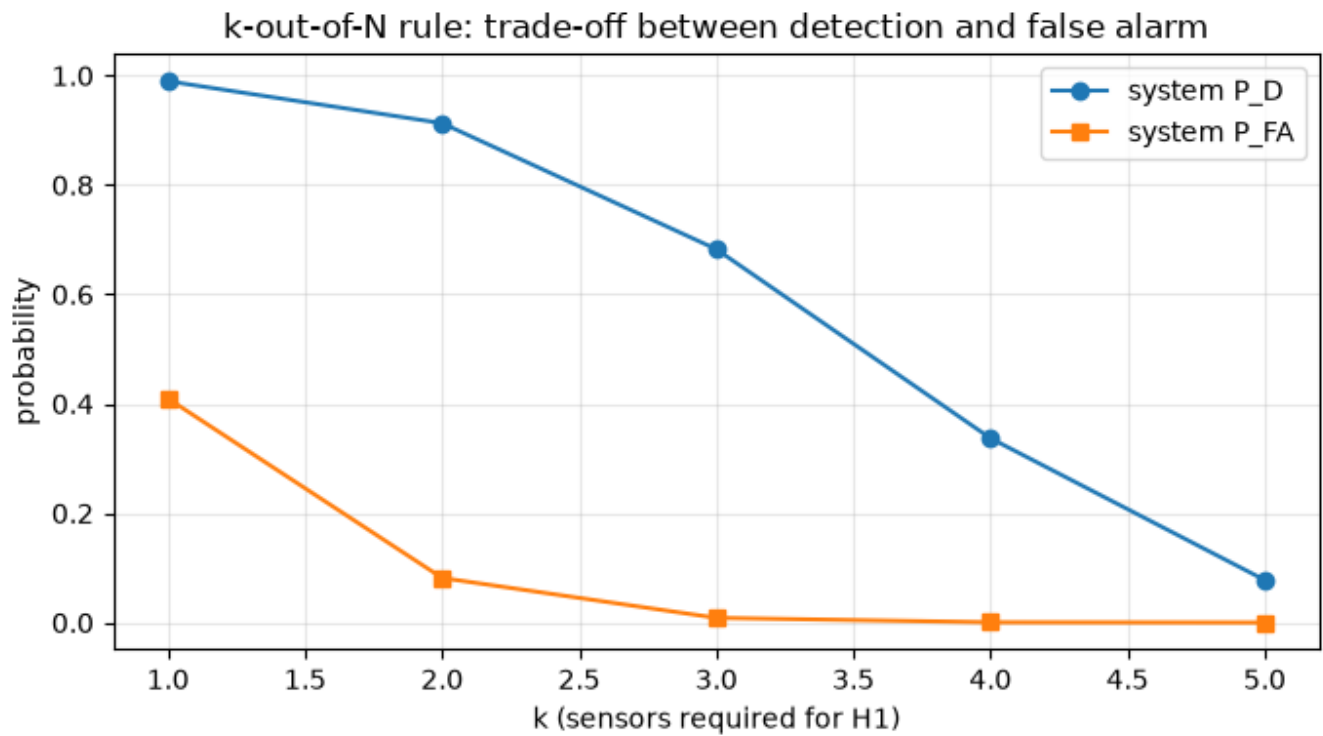
ks=range(1,N+1)
plt.figure(figsize=(7,4))
plt.plot(list(ks),[kN(pd,N,k) for k in ks],'o-',label='system P_D')
plt.plot(list(ks),[kN(pfa,N,k) for k in ks],'s-',label='system P_FA')
plt.xlabel('k (sensors required for H1)'); plt.ylabel('probability')
plt.title('k-out-of-N rule: trade-off between detection and false alarm'); plt.legend();
plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
```

single sensor: P_D=0.6, P_FA=0.1

OR (1/5) P_D=0.990 P_FA=0.4095

majority (3/5) P_D=0.683 P_FA=0.0086

AND (5/5) P_D=0.078 P_FA=0.0000



C. The optimal Chair-Varshney fusion rule

When each sensor's

$$(P_{D,i}, P_{FA,i})$$

are known, the optimal LRT fusion rule weights each local decision

$$u_i \in \{0, 1\}$$

by its **reliability**:

$$\sum_i \left[u_i \ln \frac{P_{D,i}}{P_{FA,i}} + (1 - u_i) \ln \frac{1 - P_{D,i}}{1 - P_{FA,i}} \right] \lessgtr \ln \frac{P_0}{P_1}$$

More reliable sensors receive larger weight — unlike an 'equal' majority vote.

```
def chair_varshney(u, PD, PFA, P1=0.5):
    P0=1-P1; s=0.0
    for ui,pd,pf in zip(u,PD,PFA):
        s += np.log(pd/pf) if ui==1 else np.log((1-pd)/(1-pf))
    return 1 if s >= np.log(P0/P1) else 0

# 5 heterogeneous sensors: some reliable, some noisy
PD =np.array([0.95,0.9,0.6,0.55,0.7])
PFA=np.array([0.02,0.05,0.3,0.4,0.2])
def majority(u): return 1 if sum(u)>=3 else 0

trials=20000; H1prob=0.5; cv_ok=maj_ok=0
for _ in range(trials):
    H=int(rng.random()<H1prob)
    u=[int(rng.random()<(PD[i] if H else PFA[i])) for i in range(5)]
    cv_ok += chair_varshney(u,PD,PFA)==H
```



```

maj_ok+= majority(u)==H
print(f'majority-vote accuracy    : {maj_ok/trials:.1%}')
print(f'Chair-Varshney accuracy  : {cv_ok/trials:.1%} (weights reliability -> better with
heterogeneous sensors)')

```

majority-vote accuracy : 93.5%

Chair-Varshney accuracy : 97.8% (weights reliability -> better with heterogeneous sensors)

D. Broader context: consensus and confidence ¶

- **Consensus / gossip** — nodes in a network without a center converge to a global estimate through local exchanges only.
- **Quantization** — sending a few bits instead of full values; a bandwidth-vs-accuracy trade-off.
- **Unreliable communication** — message loss/delay requires robust fusion. All of these are distributed generalizations of the same principle: **accumulating information reduces uncertainty** (the fusion equation, notebook 01).

Summary ¶

- **ROC** characterizes each sensor; the SNR sets the performance limit.
- **k-out-of-N** rules (OR/majority/AND) give different trade-offs between

$$P_D$$

and

$$P_{FA}$$

.

- **Chair-Varshney** is the optimal fusion rule — weighting by reliability, superior to majority voting when sensors are heterogeneous.

09 · High-level Fusion

Companion to the lecture: High-level Fusion — Erik Blasch, FUSION Boot Camp 2026 (lecture 7).

The slides for lecture 7 are **not** in the PDF, but Blasch's introduction (notebook 01) covers the foundations. This notebook is based on them and on the literature (Blasch, Bossé, Lambert, High-Level Information Fusion).

Levels **L0/L1** (notebooks 02–05) deal with **objects**: where and what. Levels **L2–L5** deal with **meaning**: what the situation is, what the threat/impact is, how to task sensors (process refinement), and what the human needs (user refinement).

A. Recap — the higher JDL/DFIG levels

Level Name	Question	Techniques
L2 Situation Assessment	what are the relations among entities?	Bayesian nets, graphs, ontologies
L3 Threat / Impact	what are the consequences and intent?	game theory, abductive inference
L4 Process Refinement	how to task the sensors?	sensor management (notebook 08)
L5 User Refinement	what should the human see/decide?	HMI, visualization, explanation

Distinguishing feature: at L2+ the inference is **abductive** (the most plausible explanation) and sometimes **soft** (text/human reports), unlike the quantitative/hard inference at the lower levels.

B. Threat / Impact Assessment — a numerical example

We demonstrate **kinematic + identity** fusion for threat ranking. For each target we compute a threat score combining:

- **CPA** (Closest Point of Approach) — how close the trajectory will pass to the protected asset (kinematic, from L1).
- **TTI** (Time-To-Intercept) — when (urgency).
- **P(hostile)** — the hostility probability from classification (notebook 07). Threat score:

$$\text{threat} = P(\text{hostile}) \cdot f_{CPA} \cdot f_{TTI}$$

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(4)
asset = np.array([0.,0.]) # the protected asset at the origin
```

```

def cpa_tti(pos, vel):
    """Closest Point of Approach (distance) and time to it, relative to the asset at
    origin."""
    r=pos-asset; v=vel
    if v@v < 1e-9: return np.linalg.norm(r), np.inf
    t=-(r@v)/(v@v); t=max(t,0.0)
    cpa=np.linalg.norm(r+v*t)
    return cpa, t

# five targets: position, velocity, hostility probability
tgts=[
    dict(name='T1', pos=np.array([100.,5.]), vel=np.array([-10.,0.]), p_host=0.9),
    dict(name='T2', pos=np.array([80.,80.]), vel=np.array([-8.,-8.]), p_host=0.5),
    dict(name='T3', pos=np.array([200.,2.]), vel=np.array([-5.,0.]), p_host=0.95),
    dict(name='T4', pos=np.array([30.,60.]), vel=np.array([0.,-2.]), p_host=0.2),
    dict(name='T5', pos=np.array([60.,3.]), vel=np.array([-12.,0.]), p_host=0.7),
]
for t in tgts:
    cpa,tti=cpa_tti(t['pos'],t['vel'])
    f_cpa=np.exp(-cpa/20.0)          # closer -> more dangerous
    f_tti=np.exp(-tti/15.0)         # sooner -> more urgent
    t['cpa'],t['tti'],t['threat']=cpa,tti,t['p_host']*f_cpa*f_tti

ranked=sorted(tgts,key=lambda d:-d['threat'])
print('threat ranking (L3):')
for r,t in enumerate(ranked,1):
    print(f"{r}. {t['name']} threat={t['threat']:.3f} CPA={t['cpa']:.5.1f} TTI={t['tti']:.5.1f} P(host)={t['p_host']:.3f}")

```

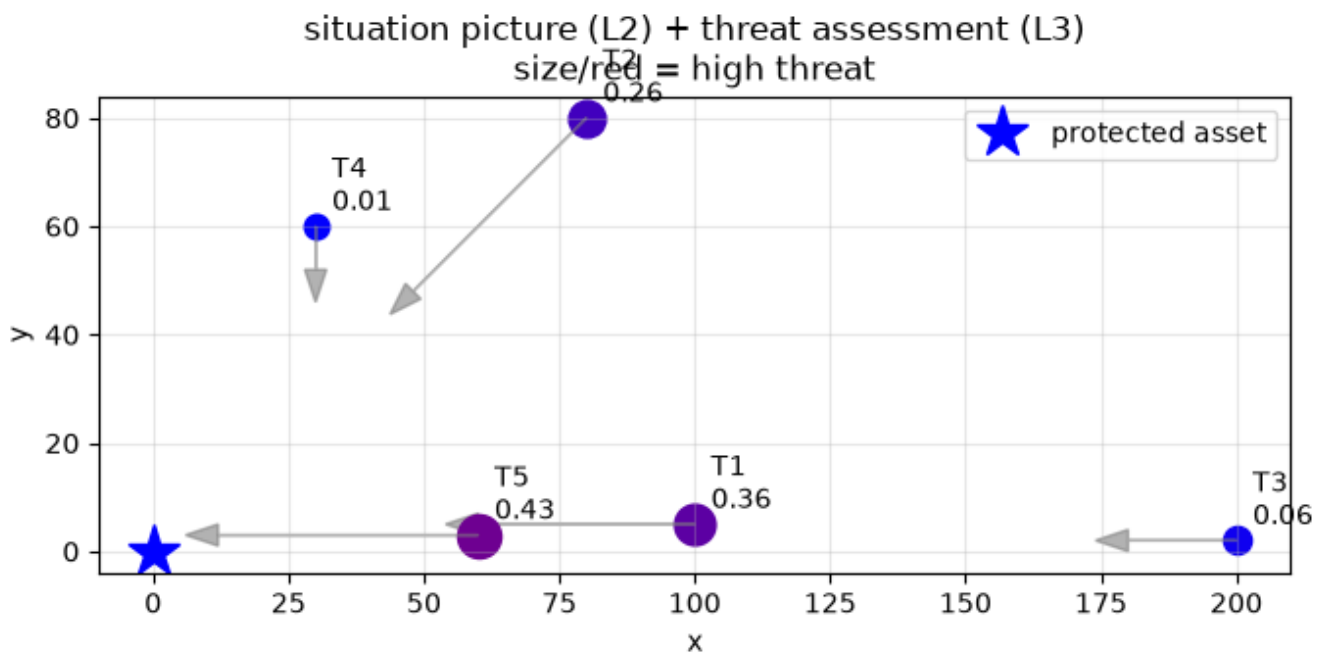
threat ranking (L3):

1.	T5	threat=0.432	CPA= 3.0	TTI= 5.0	P(host)=0.7
2.	T1	threat=0.360	CPA= 5.0	TTI= 10.0	P(host)=0.9
3.	T2	threat=0.257	CPA= 0.0	TTI= 10.0	P(host)=0.5
4.	T3	threat=0.060	CPA= 2.0	TTI= 40.0	P(host)=0.95
5.	T4	threat=0.006	CPA= 30.0	TTI= 30.0	P(host)=0.2

```

# visualization of the situation picture (L2) and threat ranking (L3)
plt.figure(figsize=(7,6))
plt.plot(*asset,'b*',ms=20,label='protected asset')
for t in tgts:
    p,v=t['pos'],t['vel']
    plt.scatter(*p,s=80+400*t['threat'],c=[[t['threat'],0,1-t['threat']]])
    plt.arrow(p[0],p[1],v[0]*4,v[1]*4,head_width=4,color='gray',alpha=.6)
    plt.annotate(f"{t['name']}\n{t['threat']:.2f}",p,textcoords='offset points',xytext=(6,6))
plt.title('situation picture (L2) + threat assessment (L3)\nsize/red = high threat');
plt.gca().set_aspect('equal')
plt.xlabel('x'); plt.ylabel('y'); plt.legend(); plt.grid(alpha=.3); plt.tight_layout();
plt.show()

```

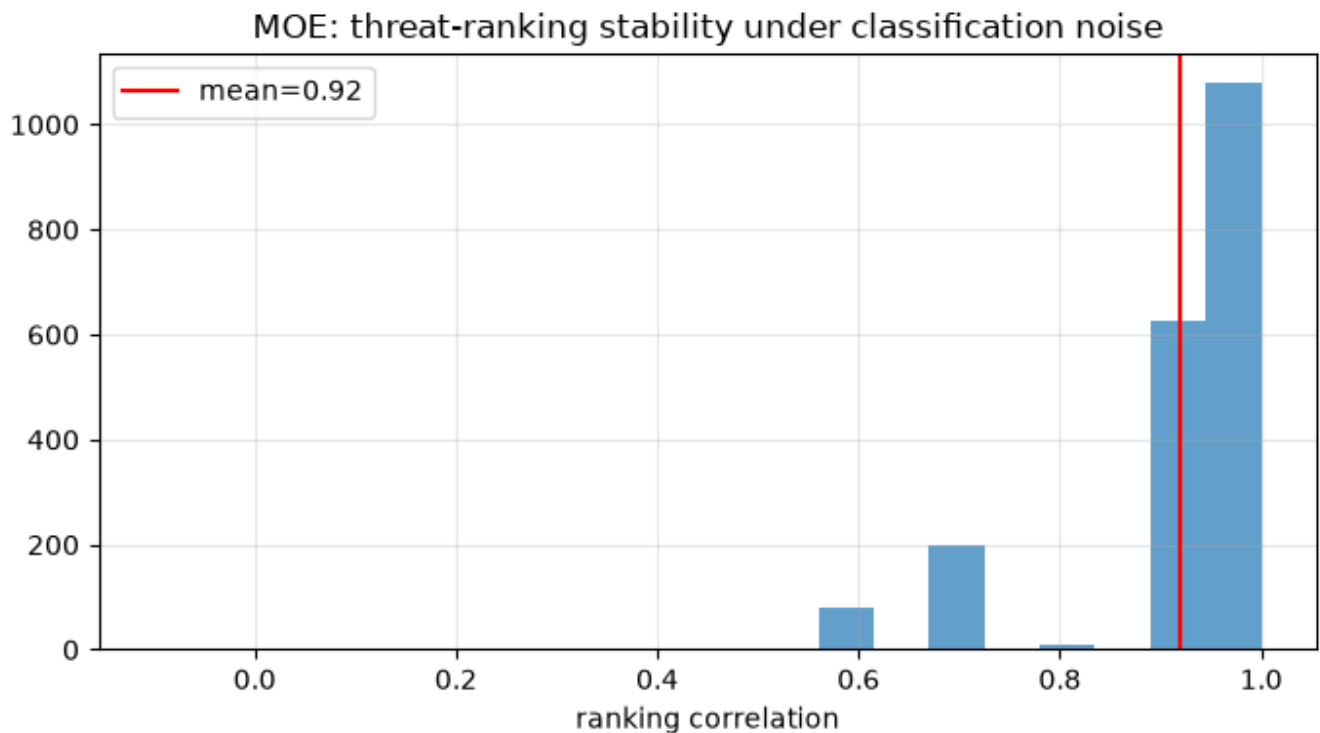


C. Evaluation measures (MOP/MOE) — the difficulty Blasch emphasized

Blasch stresses that **evaluating fusion systems is hard** (complexity, robustness, interoperability). We distinguish:

- **MOP (Measures of Performance)** — low-level technical measures: RMSE, P_D/P_{FA} , runtime.
- **MOE (Measures of Effectiveness)** — whether the mission was accomplished: threat-ranking accuracy, timely warning, operator load. A common decision-quality measure: the **area under the ROC (AUC)**.

```
# MOE: does our threat ranking correlate with the 'true' threat? (Monte-Carlo with
classification noise)
def trial():
    order_est=[]; order_true=[]
    for t in tgts:
        p_obs=np.clip(t['p_host']+0.2*rng.standard_normal(),0,1) # noisy classification
        f_cpa=np.exp(-t['cpa']/20); f_tti=np.exp(-t['tti']/15)
        order_est.append(p_obs*f_cpa*f_tti)
        order_true.append(t['threat'])
    # Spearman-like: agreement of the priority ordering
    re=np.argsort(np.argsort(-np.array(order_est)))
    rt=np.argsort(np.argsort(-np.array(order_true)))
    return np.corrcoef(re,rt)[0,1]
corrs=[trial() for _ in range(2000)]
plt.figure(figsize=(7,4)); plt.hist(corrs,bins=20,alpha=.7)
plt.axvline(np.mean(corrs),color='r',label=f'mean={np.mean(corrs):.2f}')
plt.title('MOE: threat-ranking stability under classification noise'); plt.xlabel('ranking
correlation'); plt.legend()
plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



D. Hard + Soft Fusion and the human in the loop (L5)¶

High-level fusion combines **hard** sources (physical sensors) with **soft** sources (human reports, text, intelligence). Blasch emphasizes that fusion does **not** replace human reasoning but supports it (L5): display, explanation, and interaction. The open challenges he listed in the introduction: modeling **intent**, an **intelligent reactive** adversary, and the need for **common challenge problems** and standardized evaluation for comparison and reproducibility.

Summary¶

- Levels **L2-L5** move from 'objects' to 'meaning': situation, threat, process management, and the human in the loop.
- Threat assessment combines **kinematics (CPA/TTI)** with **identity (P(hostile))** — connecting notebooks 03-07.
- Distinguish **MOP** (technical performance) from **MOE** (mission success); high-level fusion evaluation remains hard.
- The goal: **hard+soft fusion** supporting human reasoning — closing the loop back to the principles in notebook 01.